CUDA C++ 最佳实践指南（中文版 v13.2）

NVIDIA CUDA C++ 最佳实践指南 (v13.2)

> 中文翻译版 | 原文: NVIDIA CUDA C++ Best Practices Guide v13.2
> 翻译日期: 2026年5月
> 说明: 本文档是 NVIDIA CUDA C++ Best Practices Guide 的专业中文翻译版本。专有名词（如 CUDA、GPU、warp、thread、block、kernel 等）保留原文不译。代码示例、公式、表格结构完整保留。

---

1. 概述

CUDA C++ 最佳实践指南为编写高性能 CUDA 应用程序提供了实用指导。它涵盖了内存使用、并行执行和指令级效率方面的优化策略。本指南帮助开发者识别性能瓶颈，有效利用 GPU 架构，并运用性能分析工具来微调应用程序。它是最大化吞吐量、实现可扩展且高效的 CUDA 程序的必备资源。

2. 前言

本最佳实践指南是一本帮助开发者从 NVIDIA® CUDA® GPU 中获得最佳性能的手册。它介绍了成熟的并行化和优化技术，并解释了可以极大简化 CUDA 架构 GPU 编程的编码隐喻和惯用法。

虽然本文内容可作为参考手册使用，但您应当注意，有些主题会在不同上下文中随着各种编程和配置主题的探讨而多次出现。因此，建议初次阅读者按顺序通读本指南。这种方式将极大地帮助您理解高效的编程实践，并使您日后能更好地将本指南用作参考。

2.1. 谁应该阅读本指南？

本指南中的讨论均使用 C++ 编程语言，因此您应当能够轻松阅读 C++ 代码。

本指南引用并依赖于若干其他文档，您应当备有这些文档以供参考，所有这些文档均可从 CUDA 网站 https://docs.nvidia.com/cuda/ 免费获取。以下文档是特别重要的资源：

- CUDA Installation Guide
- CUDA C++ Programming Guide
- CUDA Toolkit Reference Manual

特别地，本指南的优化部分假定您已经成功下载并安装了 CUDA Toolkit（如果没有，请参考适用于您平台的 CUDA Installation Guide），并且您对 CUDA C++ 编程语言和环境有基本的了解（如果没有，请参考 CUDA C++ Programming Guide）。

2.2. 评估、并行化、优化、部署

本指南介绍了应用程序的评估、并行化、优化、部署（APOD）设计循环，旨在帮助应用开发者快速识别其代码中最容易从 GPU 加速中受益的部分，快速实现这一收益，并尽早开始在生成环境中利用由此带来的加速效果。

APOD 是一个循环过程：只需投入极少的初始时间，即可实现、测试并部署初步的加速效果，此时可以重新开始循环，识别进一步的优化机会，看到更多的加速效果，然后将更快版本的应用程序部署到生成环境中。

2.2.1. 评估

对于现有项目，第一步是评估应用程序，定位那些占据大部分执行时间的代码部分。掌握了这些信息后，开发者可以评估这些瓶颈的并行化潜力，并开始研究 GPU 加速。

通过理解最终用户的需求和约束，并应用 Amdahl 定律和 Gustafson 定律，开发者可以确定对已识别部分进行加速所能获得的性能提升上限。

2.2.2. 并行化

在确定了热点并完成了设定目标和期望的基础练习之后，开发者需要对代码进行并行化。根据原始代码的不同，这可能简单到调用现有的 GPU 优化库（如 cuBLAS、cuFFT 或 Thrust），也可能简单到添加几个预处理器指令作为对并行化编译器的提示。

另一方面，某些应用程序的设计需要进行一定程度的代码重构才能暴露其固有的并行性。由于即使 CPU 架构也需要暴露并行性来提升或仅维持顺序应用程序的性能，CUDA 系列的并行编程语言（CUDA C++、CUDA Fortran 等）旨在使这种并行性的表达尽可能简单，同时支持在专为最大并行吞吐量设计的 CUDA 架构 GPU 上运行。

2.2.3. 优化

在每一轮应用程序并行化完成之后，开发者可以转向优化实现以提升性能。由于可以考虑的优化非常多，对应用程序需求有良好的理解有助于使过程尽可能顺利。然而，与整个 APOD 过程一样，程序优化是一个迭代过程（识别优化机会、应用并测试优化、验证所获得的加速效果，然后重复），这意味着程序员不需要在获得良好加速效果之前花费大量时间死记硬背所有可能的优化策略。相反，策略可以在学习过程中逐步应用。

优化可以在多个层次上进行，从将数据传输与计算重叠，一直到微调浮点操作序列。现有的性能分析工具对于指导这一过程非常宝贵，因为它们可以帮助开发者建议下一步最佳的优化行动方案，并提供本指南优化部分相关章节的参考。

2.2.4. 部署

在完成应用程序一个或多个组件的 GPU 加速之后，可以将结果与原始预期进行比较。回想一下，最初的评估步骤允许开发者确定通过加速给定热点可获得的潜在加速上限。

在处理其他热点以提高总体加速效果之前，开发者应考虑将部分并行化的实现推进到生成环境中。这一点很重要，原因有多个：例如，它允许用户尽早从投资中获益（加速可能是部分的，但仍然有价

值），并且通过提供渐进式而非革命性的应用程序变更来最小化开发者和用户的风险。

2.3. 建议与最佳实践

在本指南中，针对 CUDA C++ 代码的设计和实现给出了具体建议。这些建议按优先级分类，优先级是建议效果和适用范围的综合考量。能为大多数 CUDA 应用程序带来实质性改进的操作具有**高优先级**，而仅影响非常特定场景的小优化则被赋予较低的优先级。

在实施较低优先级的建议之前，最好先确保所有相关的更高优先级建议已经得到应用。这种方法往往能为投入的时间带来最佳结果，并能避免过早优化的陷阱。

确定优先级时所用的收益和范围标准会因程序性质的不同而有所变化。在本指南中，它们代表的是典型情况。您的代码可能反映不同的优先级因素。无论这种可能性如何，在着手处理低优先级项目之前，验证没有被忽略的更高优先级建议是一个良好实践。

> 注意
> 为简洁起见，本指南中的代码示例省略了错误检查。然而，生成代码应系统地检查每个 API 调用返回的错误码，并通过调用 cudaGetLastError() 检查 kernel 启动的失败情况。

2.4. 评估您的应用程序

从超级计算机到移动电话，现代处理器越来越依赖于并行性来提供性能。核心计算单元（包含控制、算术、寄存器及通常的一些缓存）被复制若干次，并通过网络连接到内存。因此，所有现代处理器都需要并行代码才能充分利用其计算能力。

虽然处理器正在演进，向程序员暴露更细粒度的并行性，但许多现有应用程序要么是作为串行代码演进的，要么是作为粗粒度并行代码演进的（例如，将数据分解为并行处理的区域，子区域通过 MPI 共享）。为了从任何现代处理器架构（包括 GPU）中获益，第一步是评估应用程序以识别热点，确定它们是否可以被并行化，并理解当前和未来的相关工作负载。

3. 异构计算

CUDA 编程涉及在两种不同的平台上同时运行代码：一个包含一个或多个 CPU 的主机系统，以及一个或多个支持 CUDA 的 NVIDIA GPU 设备。

虽然 NVIDIA GPU 通常与图形相关，但它们也是强大的算术引擎，能够并行运行数千个轻量级线程。这一能力使它们非常适合能够利用并行执行的计算。

然而，设备基于与主机系统截然不同的设计，理解这些差异以及它们如何决定 CUDA 应用程序的性能对于高效使用 CUDA 至关重要。

3.1. 主机与设备之间的差异

主要差异在于线程模型和独立的物理内存：

线程资源

主机系统上的执行流水线只能支持有限数量的并发线程。例如，具有两个 32 核处理器的服务器只能同时运行 64 个线程（如果 CPU 支持同时多线程，则为此数的小倍数）。相比之下，CUDA 设备上最小的

并行执行单元包含 32 个线程（称为一个 warp 线程束）。现代 NVIDIA GPU 每个多处理器可支持多达 2048 个并发活跃线程（参见 CUDA C++ Programming Guide 的特性与规格部分）。在具有 80 个多处理器的 GPU 上，这导致超过 160,000 个并发活跃线程。

线程

CPU 上的线程通常是重量级实体。操作系统必须在 CPU 执行通道上换入和换出线程以提供多线程能力。因此，上下文切换（两个线程交换时）缓慢且代价高昂。相比之下，GPU 上的线程极其轻量。在典型系统中，数千个线程排队等待工作（每个 warp 包含 32 个线程）。如果 GPU 必须等待一个 warp 的线程，它只需开始执行另一个 warp 的工作。由于为所有活跃线程分配了独立的寄存器，在 GPU 线程之间切换时不需要交换寄存器或其他状态。资源保持分配给每个线程直到其完成执行。简而言之，CPU 核心被设计为每次最小化少量线程的延迟，而 GPU 被设计为处理大量并发的轻量级线程以最大化吞吐量。

RAM

主机系统和设备各自拥有自己独立的物理内存[1]。由于主机内存和设备内存是分离的，主机内存中的项目有时必须在设备内存和主机内存之间进行通信，如"什么在支持 CUDA 的设备上运行？"一节所述。

这些是 CPU 主机与 GPU 设备在并行编程方面的主要硬件差异。其他差异将在本文档的相关部分出现时进行讨论。考虑到这些差异而构建的应用程序可以将主机和设备视为一个统一的异构系统，其中每个处理单元被用来做它最擅长的工作：主机上的串行工作和设备上的并行工作。

3.2. 什么在支持 CUDA 的设备上运行？

在确定应用程序的哪些部分在设备上运行时，应考虑以下问题：

设备非常适合那些可以在大量数据元素上同时并行运行的计算。这通常涉及对大数据集（例如矩阵）进行的算术运算，其中相同的操作可以同时对数千甚至数百万个元素执行。这是在 CUDA 上获得良好性能的要求：软件必须使用大量（通常是数千或数万个）并发线程。支持大量线程并行运行源于 CUDA 使用的上述轻量级线程模型。

要使用 CUDA，必须将数据值从主机传输到设备。这些传输在性能方面代价高昂，应尽量减少（参见主机与设备之间的数据传输）。这一代价有以下几个影响：

- 操作的复杂度应能证明在主机和设备之间移动数据的代价是合理的。将数据用于少量线程的短暂使用的代码几乎不会看到性能收益。理想的情况是许多线程执行大量工作。
- 例如，将两个矩阵传输到设备以执行矩阵加法，然后将结果传输回主机，将不会实现太大的性能收益。这里的问题在于每个传输的数据元素所执行的操作数量。对于上述过程，假设矩阵大小为 $N \times N$，有 N^2 次操作（加法）和 $3N^2$ 个元素被传输，因此操作数与传输元素数之比为 1:3 或 O(1)。当这个比率更高时，更容易获得性能收益。例如，相同矩阵的矩阵乘法需要 N^3 次操作（乘加），因此操作数与传输元素数之比为 O(N)，在这种情况下，矩阵越大，性能收益越大。操作类型也是一个额外因素，因为加法的复杂度特征与三角函数等不同。在确定操作应在主机上执行还是在设备上执行时，重要的是将数据传输的开销考虑在内。

- 数据应尽可能长时间地保留在设备上。由于应尽量减少传输，对相同数据运行多个 kernel 的程序应倾向于将数据在 kernel 调用之间保留在设备上，而不是将中间结果传输到主机，然后再将其发回设备进行后续计算。因此，在前面的例子中，如果要相加的两个矩阵已经作为先前计算的结果存在于设备上，或者如果加法的结果将用于某个后续计算，那么矩阵加法应该在设备本地执行。即使一系列计算中的某个步骤在主机上执行可能更快，也应采用这种方法。即使是一个相对较慢的 kernel，如果能避免一次或多次主机内存与设备内存之间的传输，也可能是有利的。主机与设备之间的数据传输提供了更多细节，包括主机与设备之间带宽相对于设备内部带宽的测量数据。

为了获得最佳性能，设备上运行的相邻线程的内存访问应具有一定的连贯性。某些内存访问模式使硬件能够将多个数据项的读取或写入组合成一个操作。无法布局以实现合并的数据，或者没有足够局部性以有效利用 L1 或纹理缓存的数据，在 GPU 上用于计算时往往会看到较小的加速效果。一个值得注意的例外是完全随机的内存访问模式。一般来说，应该避免这种模式，因为与峰值能力相比，任何架构处理这些内存访问模式的效率都很低。然而，与基于缓存的架构（如 CPU）相比，延迟隐藏架构（如 GPU）倾向于更好地应对完全随机的内存访问模式。

4. 应用程序性能分析

4.1. 性能分析

许多代码用相对较少的代码量完成了大部分工作。使用性能分析器，开发者可以识别这些热点，并开始编制并行化候选项的列表。

4.1.1. 创建性能分析报告

对代码进行性能分析有多种可能的方法，但在所有情况下目标都是相同的：识别应用程序花费最多执行时间的函数。

> 注意
> 高优先级：为最大化开发者生产力，对应用程序进行性能分析以确定热点和瓶颈。

任何性能分析活动最重要的考虑因素是确保工作负载是真实的——即从测试中获得的信息以及基于该信息做出的决策与实际数据相关。使用不真实的工作负载可能导致次优的结果和浪费的精力，既可能让开发者针对不切实际的问题规模进行优化，也可能让开发者专注于错误的函数。

有多种工具可用于生成性能分析报告。以下示例基于 gprof，它是来自 GNU Binutils 集合的 Linux 平台开源性能分析器。

代码块

```
$ gcc -O2 -g -pg myprog.c
$ gprof ./a.out > profile.txt
Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
 33.34      0.02     0.02     7208     0.00     0.00  genTimeStep
 16.67      0.03     0.01      240     0.04     0.12  calcStats
 16.67      0.04     0.01        8     1.25     1.25  calcSummaryData
```

```
 9    16.67      0.05     0.01        7     1.43     1.43  write
10    16.67      0.06     0.01                             mcount
11     0.00      0.06     0.00      236     0.00     0.00  tzset
12     0.00      0.06     0.00      192     0.00     0.00  tolower
13     0.00      0.06     0.00       47     0.00     0.00  strlen
14     0.00      0.06     0.00       45     0.00     0.00  strchr
15     0.00      0.06     0.00        1     0.00    50.00  main
16     0.00      0.06     0.00        1     0.00     0.00  memcpy
17     0.00      0.06     0.00        1     0.00    10.11  print
18     0.00      0.06     0.00        1     0.00     0.00  profil
19     0.00      0.06     0.00        1     0.00    50.00  report
```

4.1.2. 识别热点

在上面的示例中，我们可以清楚地看到函数 `genTimeStep()` 占用了应用程序总运行时间的三分之一。这应该是我们并行化的第一个候选函数。理解扩展性讨论了我们可以从这种并行化中预期的潜在收益。

值得注意的是，上述示例中其他几个函数也占据了总运行时间的显著部分，如 `calcStats()` 和 `calcSummaryData()`。将这些函数也并行化应该会增加我们的加速潜力。然而，由于 APOD 是一个循环过程，我们可以选择在后续的 APOD 轮次中并行化这些函数，从而将任何给定轮次中的工作范围限制在一组较小的增量变更中。

4.1.3. 理解扩展性

应用程序通过在 CUDA 上运行而获得的性能收益大小完全取决于其可并行化的程度。不能充分并行化的代码应该在主机上运行，除非这样做会导致主机和设备之间的过度传输。

> 注意
> 高优先级：为了从 CUDA 获得最大收益，首先专注于寻找并行化串行代码的方法。

通过理解应用程序如何扩展，可以设定预期并规划增量并行化策略。强扩展性与 Amdahl 定律描述了强扩展性，它允许我们在固定问题规模下设定加速上限。弱扩展性与 Gustafson 定律描述了弱扩展性，其中加速是通过增大问题规模来实现的。在许多应用程序中，强扩展性和弱扩展性的组合是可取的。

4.1.3.1. 强扩展性与 Amdahl 定律

强扩展性衡量的是，对于固定的总问题规模，随着系统中添加更多处理器，求解时间如何减少。表现出线性强扩展性的应用程序，其加速比等于所使用的处理器数量。

强扩展性通常等同于 Amdahl 定律，该定律指定了通过并行化串行程序的某些部分可以预期的最大加速比。本质上，它表明一个程序的最大加速比 S 为：

$$S = \frac{1}{(1 - P) + \frac{P}{N}}$$

这里 P 是总串行执行时间中可并行化代码部分所占的比例，N 是并行代码部分运行在其上的处理器数量。

N 越大（即处理器数量越多），P/N 这个分数就越小。将 N 视为一个非常大的数字可以更简单地理解，这实质上将方程转换为 (S = 1/(1 - P))。现在，如果一个串行程序的 3/4 运行时间被并行化，那么相对于串行代码的最大加速比是 1 / (1 - 3/4) = 4。

实际上，大多数应用程序并不会表现出完美的线性强扩展性，即使它们确实表现出某种程度的强扩展性。对于大多数目的，关键点是可并行化部分 P 越大，潜在的加速越大。反之，如果 P 很小（意味着应用程序本质上不可大幅并行化），增加处理器数量 N 对提升性能的作用很小。因此，为了在固定问题规模下获得最大加速比，值得花精力增大 P，最大化可以并行化的代码量。

4.1.3.2. 弱扩展性与 Gustafson 定律

弱扩展性衡量的是，随着系统中添加更多处理器而每个处理器的固定问题规模不变（即总问题规模随处理器数量增加而增加）时，求解时间如何变化。

弱扩展性通常等同于 Gustafson 定律，该定律指出在实际中，问题规模随处理器数量扩展。因此，一个程序的最大加速比 S 为：

[S = N + (1 - P)(1 - N)]

这里 P 是总串行执行时间中可并行化代码部分所占的比例，N 是并行代码部分运行在其上的处理器数量。

另一种看待 Gustafson 定律的角度是：随着系统规模的扩大，保持不变的并非问题规模，而是执行时间。注意，Gustafson 定律假设串行执行与并行执行的比例保持不变，反映了建立和处理更大问题的额外代价。

4.1.3.3. 应用强扩展性与弱扩展性

理解哪种类型的扩展性最适用于应用程序是估算加速比的重要部分。对于某些应用程序，问题规模将保持不变，因此只有强扩展性适用。一个例子是模拟两个分子如何相互作用，其中分子大小是固定的。

对于其他应用程序，问题规模将增长以填满可用的处理器。例子包括对流体或结构进行网格或栅格建模，以及某些 Monte Carlo 模拟，其中增加问题规模可提供更高的准确性。

在理解应用程序性能分析结果之后，开发者应理解如果计算性能改变，问题规模将如何变化，然后应用 Amdahl 定律或 Gustafson 定律来确定加速比的上限。

5. 并行化您的应用程序

在确定了热点并完成了设定目标和期望的基础练习之后，开发者需要对代码进行并行化。根据原始代码的不同，这可能简单到调用现有的 GPU 优化库（如 cuBLAS、cuFFT 或 Thrust），也可能简单到添加几个预处理器指令作为对并行化编译器的提示。

另一方面，某些应用程序的设计需要进行一定程度的代码重构才能暴露其固有的并行性。由于即使 CPU 架构也需要暴露这种并行性来提升或仅维持顺序应用程序的性能，CUDA 系列的并行编程语言（CUDA C++、CUDA Fortran 等）旨在使这种并行性的表达尽可能简单，同时支持在专为最大并行吞吐量设计的 CUDA 架构 GPU 上运行。

6. 入门指南

并行化串行代码有几种关键策略。虽然如何将这些策略应用于特定应用程序的细节是一个复杂且因问题而异的话题，但这里列出的一般性主题，无论我们是将代码并行化以运行在多核 CPU 上，还是在 CUDA GPU 上使用，都同样适用。

6.1. 并行库

并行化应用程序最直接的方法是利用现有的、能够代替我们利用并行架构的库。CUDA Toolkit 包含许多经过 NVIDIA CUDA GPU 精细调优的此类库，例如 cuBLAS、cuFFT 等。

关键在于，只有当库能够很好地匹配应用程序的需求时，它们才最有用的。例如，已在使用其他 BLAS 库的应用程序通常可以很容易地切换到 cuBLAS，而几乎不涉及线性代数的应用程序对 cuBLAS 则没有太大用处。其他 CUDA Toolkit 库也是如此：cuFFT 具有与 FFTW 类似的接口等等。

同样值得注意的是 Thrust 库，它是一个并行 C++ 模板库，类似于 C++ Standard Template Library。Thrust 提供了丰富的数据并行原语集合，如 scan、sort 和 reduce，这些原语可以组合起来，以简洁、可读的源代码实现复杂的算法。通过以这些高级抽象来描述您的计算，您为 Thrust 提供了自动选择最有效实现的自由。因此，Thrust 既可用于 CUDA 应用程序的快速原型设计（此时程序员的生产力最为重要），也可用于生成环境，此时鲁棒性和绝对性能至关重要。

6.2. 并行化编译器

另一种并行化串行代码的常见方法是使用并行化编译器。通常，这意味着使用基于指令的方法，程序员使用 pragma 或其他类似的标记向编译器提供关于何处可找到并行性的提示，而无需修改或调整底层代码本身。通过向编译器暴露并行性，指令使编译器能够完成将计算映射到并行架构上的详细工作。

OpenACC 标准提供了一组编译器指令，用于指定标准 C、C++ 和 Fortran 中应从主机 CPU 卸载到连接的加速器（如 CUDA GPU）上的循环和代码区域。管理加速器设备的细节由支持 OpenACC 的编译器和运行时隐式处理。

详见 http://www.openacc.org/。

6.3. 编码以暴露并行性

对于需要超出已有并行库或并行化编译器所能提供的额外功能或性能的应用程序，像 CUDA C++ 这样与现有串行代码无缝集成的并行编程语言至关重要。

一旦我们在应用程序的性能分析评估中定位到一个热点，并确定自定义代码是最佳方法，我们就可以使用 CUDA C++ 将该部分代码中的并行性暴露为一个 CUDA kernel。然后，我们可以将这个 kernel 启动到 GPU 上并取回结果，而无需对应用程序的其余部分进行大量重写。

当应用程序的总运行时间大部分花费在少数几个相对独立的代码部分时，这种方法最为直接。更难并行化的是那些具有非常平坦性能分析轮廓的应用程序——即时间花费相对均匀地分布在代码库的广泛部分上的应用程序。对于后一类应用程序，可能需要进行一定程度的代码重构来暴露应用程序中固有

的并行性，但要记住，这种重构工作往往会使所有未来的架构受益，无论 CPU 还是 GPU，因此如果有必要，这种努力是非常值得的。

7. 获得正确答案

获得正确答案显然是所有计算的首要目标。在并行系统中，可能会遇到在传统串行编程中通常不会遇到的困难。这些问题包括线程问题、由于浮点值计算方式导致的意外值，以及由于 CPU 和 GPU 处理器运行方式差异带来的挑战。本章探讨可能影响返回数据正确性的问题，并指出适当的解决方案。

7.1. 验证

7.1.1. 参考比较

对任何现有程序的修改进行正确性验证的一个关键方面是建立某种机制，通过该机制可以将代表性输入的先前已知正确的参考输出与新结果进行比较。在每次修改之后，确保结果使用适用于特定算法的标准来匹配。有些算法期望逐位相同的结果，但这并非总是可能的，尤其是在涉及浮点运算的情况下；关于数值精度，请参见数值准确度与精度。对于其他算法，如果实现在某个小 epsilon 范围内与参考匹配，则可以认为是正确的。

注意，用于验证数值结果的过程可以很容易地扩展为同时验证性能结果。我们希望确保所做的每一次修改都是正确的，并且能够提升性能（以及提升多少）。将这些检查作为我们循环 APOD 过程的一个组成部分频繁进行，将有助于确保我们尽快达到预期结果。

7.1.2. 单元测试

上述参考比较方法的一个有用补充是将代码本身构建为易于在单元级别进行验证的方式。例如，我们可以将 CUDA kernel 编写为许多简短的 `__device__` 函数的集合，而不是一个大的单体 `__global__` 函数；每个 device 函数可以独立测试，然后将它们全部连接在一起。

例如，许多 kernel 除了实际计算之外，还有复杂的内存访问寻址逻辑。如果在引入大量计算之前单独验证寻址逻辑，那么这将简化后续的任何调试工作。（注意，CUDA 编译器会将任何不对全局内存产生写入贡献的 device 代码视为死代码而予以消除，因此要成功应用此策略，我们必须至少将某些内容写入全局内存作为寻址逻辑的结果。）

更进一步，如果将大多数函数定义为 `__host__ __device__` 而不仅仅是 `__device__` 函数，那么这些函数可以在 CPU 和 GPU 上进行测试，从而增加我们对函数正确性的信心，并确保结果中不会出现任何意外的差异。如果存在差异，那么这些差异将被及早发现，并且可以在简单函数的上下文中进行理解。

作为一个有益的副作用，这种策略将为我们提供一种减少代码重复的方法，如果我们希望在应用程序中同时包含 CPU 和 GPU 执行路径：如果 CUDA kernel 的大部分工作是在 `__host__ __device__` 函数中完成的，我们可以轻松地从主机代码和设备代码中调用这些函数，而无需重复代码。

7.2. 调试

CUDA-GDB 是 GNU Debugger 的一个移植版本，运行在 Linux 和 Mac 上；参见：https://developer.nvidia.com/cuda-gdb。

NVIDIA Nsight Visual Studio Edition 作为 Microsoft Visual Studio 的免费插件提供；参见：https://developer.nvidia.com/nsight-visual-studio-edition。

多个第三方调试器也支持 CUDA 调试；更多详情参见：https://developer.nvidia.com/debugging-solutions。

7.3. 数值准确度与精度

错误或意外的结果主要源于浮点数值的计算和存储方式导致的浮点精度问题。以下各节解释了主要关注的事项。浮点运算的其他特性在其他地方介绍，包括 CUDA C++ Programming Guide 的特性与技术规格部分，以及可在 https://developer.nvidia.com/content/precision-performance-floating-point-and-ieee-754-compliance-nvidia-gpus 获取的关于浮点精度和性能的白皮书及随附的网络研讨会。

7.3.1. 单精度与双精度

CUDA 计算能力 1.3 及更高的设备提供对双精度浮点值（即 64 位宽的值）的原生支持。使用双精度算术获得的结果通常会与使用单精度算术执行的相同操作有所不同，这是因为双精度具有更高的精度以及舍入问题。因此，务必确保比较相同精度的值，并在某个容差范围内表达结果，而不是期望它们完全相同。

7.3.2. 浮点数学不具有结合性

每个浮点算术运算都涉及一定量的舍入。因此，算术运算的执行顺序很重要。如果 A、B 和 C 是浮点值，(A+B)+C 不保证等于 A+(B+C)，而这在符号数学中是成立的。当您并行化计算时，可能会改变操作顺序，因此并行结果可能与串行结果不匹配。这一限制并非 CUDA 特有，而是浮点值并行计算的固有部分。

7.3.3. IEEE 754 遵从性

所有 CUDA 计算设备都遵循 IEEE 754 二进制浮点表示标准，但有一些小的例外。这些例外在 CUDA C++ Programming Guide 的特性与技术规格部分有详细说明，它们可能导致与主机系统上计算的 IEEE 754 值不同的结果。

一个关键差异是融合乘加（FMA）指令，它将乘法和加法操作合并为单条指令执行。其结果通常会与分别执行两个操作获得的结果略有不同。

7.3.4. x86 80 位计算

x86 处理器在执行浮点计算时可以使用 80 位双扩展精度数学。这些计算的结果通常会与在 CUDA 设备上执行的纯 64 位操作的结果有所不同。为了使值更加接近匹配，可以将 x86 主机处理器设置为使用常规的双精度或单精度（分别为 64 位和 32 位）。这可以通过 FLDCW x86 汇编指令或等效的操作系统 API 来完成。

8. 优化 CUDA 应用程序

在每一轮应用程序并行化完成之后，开发者可以转向优化实现以提升性能。由于可以考虑的优化非常多，对应用程序需求有良好的理解有助于使过程尽可能顺利。然而，与整个 APOD 过程一样，程序优化是一个迭代过程（识别优化机会、应用并测试优化、验证所获得的加速效果，然后重复），这意味着程序员不需要在获得良好加速效果之前花费大量时间死记硬背所有可能的优化策略。相反，策略可以在学习过程中逐步应用。

优化可以在多个层次上进行，从将数据传输与计算重叠，一直到微调浮点操作序列。现有的性能分析工具对于指导这一过程非常宝贵，因为它们可以帮助开发者建议下一步最佳的优化行动方案，并提供本指南优化部分相关章节的参考。

9. 性能指标

在尝试优化 CUDA 代码时，了解如何准确地测量性能以及理解带宽在性能测量中的作用是非常值得的。本章讨论如何使用 CPU 计时器和 CUDA 事件正确测量性能。然后探讨带宽如何影响性能指标，以及如何缓解它带来的一些挑战。

9.1. 计时

CUDA 调用和 kernel 执行可以使用 CPU 或 GPU 计时器进行计时。本节探讨这两种方法的功能、优势和注意事项。

9.1.1. 使用 CPU 计时器

任何 CPU 计时器都可以用来测量 CUDA 调用或 kernel 执行的已用时间。各种 CPU 计时方法的细节超出了本文档的范围，但开发者应始终了解其计时调用所提供的分辨率。

在使用 CPU 计时器时，关键是要记住许多 CUDA API 函数是异步的；也就是说，它们在完成工作之前就将控制权返回给调用 CPU 线程。所有 kernel 启动都是异步的，名称中带有 Async 后缀的内存复制函数也是一样。因此，要准确测量特定调用或一系列 CUDA 调用的已用时间，必须在启动和停止 CPU 计时器之前立即调用 `cudaDeviceSynchronize()` 来同步 CPU 线程与 GPU。`cudaDeviceSynchronize()` 阻塞调用 CPU 线程，直到该线程先前发出的所有 CUDA 调用完成。

虽然也可以将 CPU 线程与 GPU 上的特定 stream 或 event 同步，但这些同步函数不适用于对非默认 stream 中的代码进行计时。`cudaStreamSynchronize()` 阻塞 CPU 线程，直到先前发出到给定 stream 中的所有 CUDA 调用完成。`cudaEventSynchronize()` 阻塞直到 GPU 记录了特定 stream 中的一个给定 event。由于驱动程序可能交错执行来自其他非默认 stream 的 CUDA 调用，其他 stream 中的调用可能被包含在计时中。

由于默认 stream（stream 0）在设备上表现出串行化行为（默认 stream 中的操作只能在所有 stream 中的所有先前调用完成后才能开始；并且在它完成之前，任何 stream 中的后续操作都不能开始），这些函数可以可靠地用于默认 stream 中的计时。

请注意，本节中提到的 CPU 与 GPU 同步点意味着 GPU 处理流水线的停顿，因此应谨慎使用以最小化其性能影响。

9.1.2. 使用 CUDA GPU 计时器

CUDA 事件 API 提供了创建和销毁事件、记录事件（包括时间戳）以及将时间戳差值转换为以毫秒为单位的浮点值等调用。如何使用 CUDA 事件对代码计时展示了它们的用法。

如何使用 CUDA 事件对代码计时

代码块

```
cudaEvent_t start, stop;
float time;

cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord( start, 0 );
kernel<<<grid,threads>>> ( d_odata, d_idata, size_x, size_y,
                           NUM_REPS);
cudaEventRecord( stop, 0 );
cudaEventSynchronize( stop );

cudaEventElapsedTime( &time, start, stop );
cudaEventDestroy( start );
cudaEventDestroy( stop );
```

这里 `cudaEventRecord()` 用于将 start 和 stop 事件放入默认 stream（stream 0）中。设备将在 stream 中到达该事件时记录一个时间戳。`cudaEventElapsedTime()` 函数返回 start 和 stop 事件记录之间经过的时间。该值以毫秒为单位表示，分辨率约为半微秒。与此清单中的其他调用一样，它们的具体操作、参数和返回值在 CUDA Toolkit Reference Manual 中有描述。注意，计时是在 GPU 时钟上测量的，因此计时分辨率与操作系统无关。

9.2. 带宽

带宽——数据传输的速率——是性能最重要的门控因素之一。几乎所有对代码的更改都应该在它们如何影响带宽的背景下进行。如本指南的内存优化部分所述，带宽会受到数据存储所用内存选择、数据布局方式及访问顺序以及其他因素的显著影响。

为了准确测量性能，计算理论带宽和有效带宽是很有用的。当后者远低于前者时，设计或实现细节很可能降低了带宽，后续优化工作的首要目标应是提高带宽。

> 注意
> 高优先级：将计算的有效带宽作为衡量性能和优化收益的指标。

9.2.1. 理论带宽计算

理论带宽可以使用产品文档中提供的硬件规格来计算。例如，NVIDIA Tesla V100 使用 HBM2（双倍数据速率）RAM，内存时钟频率为 877 MHz，内存接口宽度为 4096 位。

使用这些数据项，NVIDIA Tesla V100 的峰值理论内存带宽为 898 GB/s：

[\left. \left(0.877 \times 10^{9} \right. \times (4096/8) \times 2 \right) \div 10^{9} = 898\text{GB/s}]

在此计算中，内存时钟频率转换为 Hz，乘以接口宽度（除以 8，将位转换为字节），再乘以 2（由于双倍数据速率）。最后，将此乘积除以 10^9 将结果转换为 GB/s。

> 注意
> 有些计算使用 1024^3 而非 10^9 进行最终计算。在这种情况下，带宽将是 836.4 GiB/s。在计算理论带宽和有效带宽时使用相同的除数非常重要，这样才能使比较有效。

> 注意
> 在启用了 ECC 的 GDDR 内存 GPU 上，可用 DRAM 减少 6.25% 以便存储 ECC 位。为每个内存事务获取 ECC 位也会使有效带宽相比同一 GPU 禁用 ECC 时降低约 20%，不过 ECC 对带宽的确切影响可能更高，并取决于内存访问模式。另一方面，HBM2 内存提供专用的 ECC 资源，允许无开销的 ECC 保护。[2]

[2]: 作为一个例外，对 HBM2 的分散写操作会受到 ECC 带来的一些开销，但比在受 ECC 保护的 GDDR5 内存上使用类似访问模式时的开销小得多。

9.2.2. 有效带宽计算

有效带宽通过对特定程序活动进行计时并了解程序如何访问数据来计算。为此，使用以下方程：

[\text{Effective\ bandwidth} = \left({\left(B_{r} + B_{w} \right) \div 10^{9}} \right) \div \text{time}]

这里，有效带宽的单位是 GB/s，(B_r) 是每个 kernel 读取的字节数，(B_w) 是每个 kernel 写入的字节数，时间以秒为单位。

例如，要计算 2048 × 2048 矩阵复制的有效带宽，可以使用以下公式：

[\text{Effective\ bandwidth} = \left({\left(2048^{2} \times 4 \times 2 \right) \div 10^{9}} \right) \div \text{time}]

元素数量乘以每个元素的大小（float 为 4 字节），乘以 2（因为有读和写），除以 10^9（或 1024^3）得到传输的 GB 内存数。此数字除以以秒为单位的时间得到 GB/s。

9.2.3. Visual Profiler 报告的吞吐量

对于计算能力 2.0 或更高的设备，Visual Profiler 可用于收集多种不同的内存吞吐量度量。以下吞吐量指标可以显示在 Details 或 Detail Graphs 视图中：

- Requested Global Load Throughput
- Requested Global Store Throughput
- Global Load Throughput
- Global Store Throughput
- DRAM Read Throughput
- DRAM Write Throughput

Requested Global Load Throughput 和 Requested Global Store Throughput 值表示 kernel 请求的全局内存吞吐量，因此对应于有效带宽计算中所示的有效带宽。

由于最小内存事务大小大于大多数字大小，kernel 实际需要的内存吞吐量可能包括传输 kernel 未使用的数据。对于全局内存访问，这个实际吞吐量由 Global Load Throughput 和 Global Store Throughput 值报告。

重要的是要注意，这两个数字都是有用的。实际内存吞吐量显示代码与硬件限制的接近程度，而有效（或请求）带宽与实际带宽的比较则很好地估计了由于内存访问合并不理想而浪费了多少带宽（参见全局内存的合并访问）。对于全局内存访问，请求内存带宽与实际内存带宽的这种比较由 Global Memory Load Efficiency 和 Global Memory Store Efficiency 指标报告。

10. 内存优化

内存优化是性能最重要的领域。目标是通过最大化带宽来最大化对硬件的利用。带宽的最佳服务方式是尽可能使用更多的高速内存和尽可能少的慢速访问内存。本章讨论主机和设备上的各种内存类型，以及如何最好地设置数据项以高效使用内存。

10.1. 主机与设备之间的数据传输

设备内存与 GPU 之间的峰值理论带宽远高于主机内存与设备内存之间的峰值理论带宽（例如，NVIDIA Tesla V100 上分别为 898 GB/s 与 PCIe x16 Gen3 上的 16 GB/s）。因此，为了获得最佳的整体应用程序性能，最小化主机与设备之间的数据传输至关重要，即使这意味着在 GPU 上运行那些与在主机 CPU 上运行相比并未显示出任何加速效果的 kernel。

> 注意
> 高优先级：最小化主机与设备之间的数据传输，即使这意味着在设备上运行一些与在主机 CPU 上运行相比没有性能提升的 kernel。

中间数据结构应在设备内存中创建，由设备操作，并在不被主机映射或复制到主机内存的情况下销毁。

此外，由于每次传输都有相关开销，将许多小传输批量合并为一个大传输比分别进行每次传输性能要好得多，即使这样做需要将非连续的内存区域打包到连续缓冲区中，然后在传输之后解包。

最后，使用页锁定（或 pinned）内存时，主机与设备之间可以获得更高的带宽，如 CUDA C++ Programming Guide 和本文档的 Pinned Memory 部分所述。

10.1.1. Pinned 内存

页锁定或 pinned 内存传输可实现主机与设备之间的最高带宽。例如，在 PCIe x16 Gen3 卡上，pinned 内存可以达到大约 12 GB/s 的传输速率。

Pinned 内存使用 Runtime API 中的 `cudaHostAlloc()` 函数分配。bandwidthTest CUDA Sample 展示了如何使用这些函数以及如何测量内存传输性能。

对于已经预先分配的系统内存区域，可以使用 `cudaHostRegister()` 即时 pin 内存，而无需分配单独的缓冲区并复制数据。

Pinned 内存不应过度使用。过度使用会降低整体系统性能，因为 pinned 内存是一种稀缺资源，但多少才算太多很难预先知道。此外，与大多数正常的系统内存分配相比，系统内存的 pin 操作是一个重量级操作，因此与所有优化一样，应针对应用程序及其运行的系统进行测试，以获得最佳性能参数。

10.1.2. 异步传输及其与计算的重叠

使用 `cudaMemcpy()` 在主机和设备之间进行数据传输是阻塞传输；也就是说，只有在数据传输完成后，控制权才返回给主机线程。`cudaMemcpyAsync()` 函数是 `cudaMemcpy()` 的非阻塞变体，其中控制权立即返回给主机线程。与 `cudaMemcpy()` 相比，异步传输版本需要 pinned 主机内存（参见 Pinned Memory），并且包含一个额外参数，即 stream ID。Stream 只是设备上按顺序执行的一系列操作。不同 stream 中的操作可以交错执行，并且在某些情况下可以重叠——这一特性可用于隐藏主机和设备之间的数据传输。

异步传输以两种不同的方式实现数据传输与计算的重叠。在所有支持 CUDA 的设备上，可以实现主机计算与异步数据传输以及设备计算的重叠。例如，异步传输及其与计算的重叠展示了例程 `cpuFunction()` 中的主机计算如何在数据被传输到设备并且使用设备的 kernel 正在执行时进行。

重叠计算和数据传输

代码块

```
cudaMemcpyAsync(a_d, a_h, size, cudaMemcpyHostToDevice, 0);
kernel<<<grid, block>>>(a_d);
cpuFunction();
```

`cudaMemcpyAsync()` 函数的最后一个参数是 stream ID，本例中使用默认 stream（stream 0）。kernel 也使用默认 stream，因此在内存复制完成之前 kernel 不会开始执行；因此，不需要显式同步。由于内存复制和 kernel 都立即将控制权返回给主机，主机函数 `cpuFunction()` 与它们的执行重叠。

在"异步传输及其与计算的重叠"的示例中，内存复制和 kernel 执行是顺序进行的。在能够并发复制和计算的设备上，可以实现设备上的 kernel 执行与主机和设备之间的数据传输相重叠。设备是否具有此能力由 `cudaDeviceProp` 结构的 `asyncEngineCount` 字段指示（或在 deviceQuery CUDA Sample 的输出中列出）。在具有此能力的设备上，重叠再次需要 pinned 主机内存，此外，数据传输和 kernel 必须使用不同的非默认 stream（具有非零 stream ID 的 stream）。对于这种重叠，必须使用非默认 stream，因为内存复制、内存设置函数以及使用默认 stream 的 kernel 调用只有在设备上所有先前的调用（在任何 stream 中）完成后才开始，并且在它们完成之前，设备上的任何操作（在任何 stream 中）都不会开始。

异步传输及其与计算的重叠说明了基本技术。

并发复制和执行

代码块

```
cudaStreamCreate(&stream1);
```

```
2   cudaStreamCreate(&stream2);
3   cudaMemcpyAsync(a_d, a_h, size, cudaMemcpyHostToDevice, stream1);
4   kernel<<<grid, block, 0, stream2>>>(otherData_d);
```

在此代码中，创建了两个 stream，并在 `cudaMemcpyAsync` 调用的最后一个参数和 kernel 的执行配置中指定用于数据传输和 kernel 执行。

异步传输及其与计算的重叠演示了如何将 kernel 执行与异步数据传输重叠。当数据依赖关系允许将数据分块并在多个阶段传输，启动多个 kernel 在每块数据到达时对其进行操作时，可以使用此技术。顺序复制和执行与分阶段并发复制和执行演示了这一点。它们产生等效的结果。第一段显示参考顺序实现，它传输并操作一个包含 N 个 float 的数组（假设 N 可以被 nThreads 整除）。

顺序复制和执行

代码块

```
1   cudaMemcpy(a_d, a_h, N*sizeof(float), dir);
2   kernel<<<N/nThreads, nThreads>>>(a_d);
```

分阶段并发复制和执行展示了如何将传输和 kernel 执行拆分为 nStreams 个阶段。这种方法允许数据传输和执行的部分重叠。

分阶段并发复制和执行

代码块

```
1   size=N*sizeof(float)/nStreams;
2   for (i=0; i<nStreams; i++) {
3       offset = i*N/nStreams;
4       cudaMemcpyAsync(a_d+offset, a_h+offset, size, dir, stream[i]);
5       kernel<<<N/(nThreads*nStreams), nThreads, 0,
6                stream[i]>>>(a_d+offset);
7   }
```

（在分阶段并发复制和执行中，假设 N 可以被 `nThreads * nStreams` 整除。）由于 stream 内的执行是顺序的，在各自 stream 中的数据传输完成之前，没有一个 kernel 会启动。当前的 GPU 可以同时处理异步数据传输和执行 kernel。具有单个复制引擎的 GPU 可以执行一个异步数据传输并执行 kernel，而具有两个复制引擎的 GPU 可以同时执行一个从主机到设备的异步数据传输、一个从设备到主机的异步数据传输，并执行 kernel。GPU 上复制引擎的数量由 `cudaDeviceProp` 结构的 `asyncEngineCount` 字段给出，该字段也列在 deviceQuery CUDA Sample 的输出中。（需要指出的是，不可能将阻塞传输与异步传输重叠，因为阻塞传输发生在默认 stream 中，因此在所有先前的 CUDA 调用完成之前不会开始。在它完成之前，它不会允许任何其他 CUDA 调用开始。）图 1 给出了两个代码段执行时间线的对比图，对于分阶段并发复制和执行（图的底部），nStreams 等于 4。

图 1 复制和 kernel 执行的时间线对比

- 顶部：顺序
- 底部：并发

在此示例中，假设数据传输时间和 kernel 执行时间相当。在这种情况下，当执行时间 ((t_E)) 超过传输时间 ((t_T)) 时，对于分阶段版本，总时间的粗略估计为 (t_E + t_T / nStreams)，而顺序版本为 (t_E + t_T)。如果传输时间超过执行时间，总时间的粗略估计为 (t_T + t_E / nStreams)。

10.1.3. Zero Copy

Zero copy 是 CUDA Toolkit 2.2 版本中新增的一个特性。它使 GPU 线程能够直接访问主机内存。为此，它需要映射的 pinned（不可分页）内存。在集成 GPU（即 CUDA 设备属性结构的 `integrated` 字段设置为 1 的 GPU）上，映射的 pinned 内存始终是一种性能增益，因为它避免了多余的复制，因为集成 GPU 和 CPU 内存在物理上是相同的。在独立 GPU 上，映射的 pinned 内存仅在某些情况下是有利的。由于数据不在 GPU 上缓存，映射的 pinned 内存应只读取或写入一次，并且读写的全局加载和存储操作应该是合并的。Zero copy 可以替代 stream 使用，因为 kernel 发起的数据传输会自动与 kernel 执行重叠，而无需设置和确定最优 stream 数量的开销。

> 注意
> 低优先级：对于 CUDA Toolkit 2.2 及更高版本，在集成 GPU 上使用 zero-copy 操作。

Zero-copy 主机代码中的主机代码展示了 zero copy 的典型设置方式。

Zero-copy 主机代码

代码块

```
float *a_h, *a_map;
...
cudaGetDeviceProperties(&prop, 0);
if (!prop.canMapHostMemory)
    exit(0);
cudaSetDeviceFlags(cudaDeviceMapHost);
cudaHostAlloc(&a_h, nBytes, cudaHostAllocMapped);
cudaHostGetDevicePointer(&a_map, a_h, 0);
kernel<<<gridSize, blockSize>>>(a_map);
```

在此代码中，`cudaGetDeviceProperties()` 返回结构的 `canMapHostMemory` 字段用于检查设备是否支持将主机内存映射到设备的地址空间。通过调用 `cudaSetDeviceFlags()` 并传入 `cudaDeviceMapHost` 来启用页锁定内存映射。注意，`cudaSetDeviceFlags()` 必须在设置设备或进行需要状态的 CUDA 调用（即基本上在创建 context 之前）之前调用。页锁定映射主机内存使用 `cudaHostAlloc()` 分配，映射设备地址空间的指针通过函数 `cudaHostGetDevicePointer()` 获取。在 Zero-copy 主机代码的代码中，`kernel()` 可以使用指针 `a_map` 引用映射的 pinned 主机内存，其方式与 `a_map` 指向设备内存中的位置完全相同。

> 注意
> 映射的 pinned 主机内存允许您将 CPU-GPU 内存传输与计算重叠，同时避免使用 CUDA stream。但由于对此类内存区域的任何重复访问都会导致重复的 CPU-GPU 传输，请考虑在设备内存中创建第二个区域，手动缓存先前读取的主机内存数据。

10.1.4. 统一虚拟寻址（UVA）

计算能力 2.0 及更高版本的设备在 64 位 Linux 和 Windows 上支持一种称为统一虚拟寻址（Unified Virtual Addressing，UVA）的特殊寻址模式。通过 UVA，主机内存和所有已安装受支持设备的设备内存共享一个单一的虚拟地址空间。

在 UVA 之前，应用程序必须为每个指针跟踪哪些指针指向设备内存（以及哪个设备），哪些指向主机内存，作为单独的元数据（或作为程序中硬编码的信息）。而使用 UVA，可以通过使用 `cudaPointerGetAttributes()` 检查指针的值来简单地确定指针指向的物理内存空间。

在 UVA 下，使用 `cudaHostAlloc()` 分配的 pinned 主机内存将具有相同的主机和设备指针，因此无需为此类分配调用 `cudaHostGetDevicePointer()`。然而，通过 `cudaHostRegister()` 事后 pin 的主机内存分配将继续具有与其主机指针不同的设备指针，因此在这种情况下 `cudaHostGetDevicePointer()` 仍然必要。

UVA 也是在受支持的 GPU 和受支持的配置中，通过 PCIe 总线或 NVLink 直接进行点对点（P2P）数据传输（绕过主机内存）的必要前提。

有关 UVA 和 P2P 的进一步说明和软件要求，请参见 CUDA C++ Programming Guide。

10.2. 设备内存空间

CUDA 设备使用多种内存空间，这些内存空间具有不同的特性，反映了它们在 CUDA 应用程序中的不同用途。这些内存空间包括 global、local、shared、texture 和 registers，如 图 2 所示。

图 2 CUDA 设备上的内存空间

在这些不同的内存空间中，global memory 是最丰富的；关于每个计算能力级别各种内存空间可用的内存量，请参见 CUDA C++ Programming Guide 的特性与技术规格部分。Global、local 和 texture 内存具有最大的访问延迟，其次是 constant memory、shared memory 和 register file。

各种内存类型的主要特征如 表 1 所示。

表 1 设备内存的显著特征

Memory	Location on/off chip	Cached	Access	Scope	Lifetime
Register	On	n/a	R/W	1 thread	Thread
Local	Off	Yes † †	R/W	1 thread	Thread

Shared	On	n/a	R/W	All threads in block	Block
Global	Off	†	R/W	All threads + host	Host allocation
Constant	Off	Yes	R	All threads + host	Host allocation
Texture	Off	Yes	R	All threads + host	Host allocation

† 在计算能力 6.0 和 7.x 的设备上默认缓存于 L1 和 L2；在较低计算能力的设备上默认仅缓存于 L2，但某些设备允许通过编译标志选择加入 L1 缓存。

† † 除计算能力 5.x 的设备外，默认缓存于 L1 和 L2；计算能力 5.x 的设备仅将局部变量缓存于 L2。

对于纹理访问，如果纹理引用绑定到全局内存中的线性数组，则设备代码可以写入底层数组。绑定到 CUDA 数组的纹理引用可以通过将 surface 绑定到相同的底层 CUDA 数组存储来通过 surface-write 操作进行写入。应避免在同一个 kernel 启动中读取纹理的同时写入其底层全局内存数组，因为纹理缓存是只读的，并且当关联的全局内存被修改时不会失效。

10.2.1. 全局内存的合并访问

在为 CUDA 架构 GPU 编程时，一个非常重要的性能考虑因素是全局内存访问的合并。一个 warp 中线程的全局内存加载和存储被设备合并为尽可能少的事务。

10. 内存优化（续）

10.2.1.2 顺序但未对齐的访问模式

如果 warp 中的线程按顺序访问连续但未与 32 字节段对齐的内存，则会请求五个 32 字节段，如图 4 所示。

Figure 4: 跨越五个 32 字节段的未对齐顺序地址

通过 CUDA Runtime API 分配的内存（例如通过 cudaMalloc() 分配）保证至少对齐到 256 字节。因此，选择合理的线程块大小（例如 warp 大小的倍数，即当前 GPU 上为 32），有助于 warp 进行正确对齐的内存访问。（考虑一下如果线程块大小不是 warp 大小的倍数，第二个、第三个及后续线程块所访问的内存地址会发生什么情况。）

10.2.1.3 未对齐访问的影响

使用一个简单的拷贝 kernel 来探索未对齐访问的影响是简单且有益的，如下面的代码所示。
A copy kernel that illustrates misaligned accesses：

代码块

```
__global__ void offsetCopy(float *odata, float* idata, int offset)
{
    int xid = blockIdx.x * blockDim.x + threadIdx.x + offset;
    odata[xid] = idata[xid];
}
```

在上面的 kernel 中，数据从输入数组 idata 复制到输出数组，两者都位于 global memory 中。kernel 在主机代码的循环中执行，参数 offset 从 0 变化到 32（例如，图 4 对应了这些未对齐情况）。在 NVIDIA Tesla V100（计算能力 7.0）上，不同偏移量下拷贝的有效带宽如图 5 所示。

Figure 5: offsetCopy kernel 的性能

对于 NVIDIA Tesla V100，无偏移或偏移量为 8 个字的倍数时，global memory 访问会产生四次 32 字节事务，实现带宽约为 790 GB/s。否则，每个 warp 会加载五个 32 字节段，我们预期吞吐量约为无偏移时的 4/5。然而，在这个特定例子中，偏移情况下的实际吞吐量约为 9/10，因为相邻的 warp 会重用相邻 warp 已获取的缓存行。因此，虽然影响仍然明显，但不如我们预期的那么大。如果相邻 warp 对过度获取的缓存行复用程度不高，影响会更加显著。

10.2.1.4 跨步访问

如上所述，在顺序但未对齐的访问情况下，缓存有助于减轻性能影响。然而，对于非单位跨步访问，情况可能有所不同——这种模式在处理多维数据或矩阵时经常出现。因此，确保尽可能多地利用每个已获取缓存行中的数据，是对这些设备进行内存访问性能优化的重要组成部分。

为了说明跨步访问对有效带宽的影响，请参考跨步拷贝 kernel strideCopy()，它以 stride 为跨步在 idata 和 odata 之间在线程间复制数据。
A kernel to illustrate non-unit stride data copy：

代码块

```
__global__ void strideCopy(float *odata, float* idata, int stride)
{
    int xid = (blockIdx.x*blockDim.x + threadIdx.x)*stride;
    odata[xid] = idata[xid];
}
```

图 6 说明了这种情况：warp 中的线程以跨步 2 访问内存中的字。这导致在 Tesla V100（计算能力 7.0）上每个 warp 需要加载八个 L2 缓存段。

Figure 6: 相邻线程以跨步 2 访问内存

跨步为 2 时，加载/存储效率为 50%，因为事务中一半的元素未被使用，造成带宽浪费。随着跨步增大，有效带宽不断下降，直到一个 warp 中的 32 个线程需要加载 32 个 32 字节段，如图 7 所示。

Figure 7: strideCopy kernel 的性能

如图 7 所示，应尽可能避免非单位跨步的 global memory 访问。一种解决方法利用了 shared memory，这将在下一节中讨论。

10.2.2 L2 缓存

从 CUDA 11.0 开始，计算能力 8.0 及以上的设备能够影响数据在 L2 缓存中的持久性。由于 L2 缓存是片上缓存，它潜在地为 global memory 访问提供更高的带宽和更低的延迟。更多详情请参考《CUDA C++ Programming Guide》中的 L2 访问管理部分。

10.2.2.1 L2 缓存访问窗口

当 CUDA kernel 重复访问 global memory 中的某个数据区域时，此类数据访问可被视为持久性访问。反之，如果数据仅访问一次，此类数据访问可被视为流式访问。L2 缓存的一部分可以被预留用于对 global memory 中数据区域的持久性访问。如果这部分预留空间未被持久性访问使用，则流式或普通数据访问可以使用它。

可以调整用于持久性访问的 L2 缓存预留大小，但有一定限制：

代码块

```
cudaGetDeviceProperties(&prop, device_id);
cudaDeviceSetLimit(cudaLimitPersistingL2CacheSize,
prop.persistingL2CacheMaxSize); /* Set aside max possible size of L2 cache for
persisting accesses */
```

用户数据到 L2 预留部分的映射可以通过 CUDA stream 或 CUDA graph kernel 节点上的访问策略窗口来控制。以下示例展示了如何在 CUDA stream 上使用访问策略窗口。

代码块

```
cudaStreamAttrValue stream_attribute;
// Stream level attributes data structure
stream_attribute.accessPolicyWindow.base_ptr  = reinterpret_cast<void*>(ptr);
// Global Memory data pointer
stream_attribute.accessPolicyWindow.num_bytes = num_bytes;
// Number of bytes for persisting accesses.

// (Must be less than cudaDeviceProp::accessPolicyMaxWindowSize)
stream_attribute.accessPolicyWindow.hitRatio  = 1.0;
// Hint for L2 cache hit ratio for persisting accesses in the num_bytes region
stream_attribute.accessPolicyWindow.hitProp   = cudaAccessPropertyPersisting;
// Type of access property on cache hit
stream_attribute.accessPolicyWindow.missProp  = cudaAccessPropertyStreaming;
// Type of access property on cache miss.

//Set the attributes to a CUDA stream of type cudaStream_t
```

```
cudaStreamSetAttribute(stream, cudaStreamAttributeAccessPolicyWindow,
&stream_attribute);
```

访问策略窗口需要设置 hitRatio 和 num_bytes 的值。根据 num_bytes 参数的值和 L2 缓存的大小，可能需要调整 hitRatio 的值以避免 L2 缓存行的抖动。

10.2.2.2 调整访问窗口命中率

hitRatio 参数可用于指定获得 hitProp 属性的访问比例。例如，如果 hitRatio 值为 0.6，则 global memory 区域 [ptr..ptr+num_bytes) 中 60% 的内存访问具有持久性属性，40% 具有流式属性。为了理解 hitRatio 和 num_bytes 的效果，我们使用一个滑动窗口微基准测试。

该微基准测试使用 GPU global memory 中 1024 MB 的区域。首先，我们使用 cudaDeviceSetLimit() 为持久性访问预留 30 MB 的 L2 缓存，如上所述。然后，如下图所示，我们指定内存区域前 freqSize * sizeof(int) 字节的访问为持久性访问，这些数据将使用 L2 预留部分。在我们的实验中，我们将持久数据区域的大小从 10 MB 变化到 60 MB，以模拟数据适配或超出可用 30 MB L2 预留部分的各种场景。请注意，NVIDIA Tesla A100 GPU 总共有 40 MB 的 L2 缓存容量。内存区域中剩余数据的访问（即流式数据）被视为普通或流式访问，将使用非预留 L2 部分剩余的 10 MB（除非部分 L2 预留空间未被使用）。

Figure 8: 滑动窗口实验中将持久数据访问映射到预留 L2

考虑以下 kernel 代码和访问窗口参数，作为滑动窗口实验的实现。

代码块

```
__global__ void kernel(int *data_persistent, int *data_streaming, int
dataSize, int freqSize) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    /*Each CUDA thread accesses one element in the persistent data section
      and one element in the streaming data section.
      Because the size of the persistent memory region (freqSize * sizeof(int)
bytes) is much
      smaller than the size of the streaming memory region (dataSize *
sizeof(int) bytes), data
      in the persistent region is accessed more frequently*/

    data_persistent[tid % freqSize] = 2 * data_persistent[tid % freqSize];
    data_streaming[tid % dataSize] = 2 * data_streaming[tid % dataSize];
}

stream_attribute.accessPolicyWindow.base_ptr  = reinterpret_cast<void*>
(data_persistent);
stream_attribute.accessPolicyWindow.num_bytes = freqSize * sizeof(int);
//Number of bytes for persisting accesses in range 10-60 MB
```

```
16   stream_attribute.accessPolicyWindow.hitRatio  = 1.0;
     //Hint for cache hit ratio. Fixed value 1.0
```

上述 kernel 的性能如下图表所示。当持久数据区域很好地适配入 L2 缓存的 30 MB 预留部分时，可观察到高达 50% 的性能提升。然而，一旦持久数据区域的大小超过 L2 预留缓存部分的大小，由于 L2 缓存行抖动，会观察到约 10% 的性能下降。

Figure 9: 固定命中率为 1.0 时滑动窗口基准测试的性能

为了优化性能，当持久数据的大小超过预留 L2 缓存部分的大小时，我们按如下方式调整访问窗口中的 num_bytes 和 hitRatio 参数。

代码块

```
1   stream_attribute.accessPolicyWindow.base_ptr  = reinterpret_cast<void*>
    (data_persistent);
2   stream_attribute.accessPolicyWindow.num_bytes = 20*1024*1024;
                  //20 MB
3   stream_attribute.accessPolicyWindow.hitRatio  =
    (20*1024*1024)/((float)freqSize*sizeof(int));  //Such that up to 20MB of data
    is resident.
```

我们将访问窗口中的 num_bytes 固定为 20 MB，并调整 hitRatio，使得总持久数据中的随机 20 MB 驻留在 L2 预留缓存部分中。该持久数据的剩余部分将使用流式属性进行访问。这有助于减少缓存抖动。结果如下图所示，无论持久数据是否适配 L2 预留部分，性能都表现良好。

Figure 10: 调整命中率后滑动窗口基准测试的性能

10.2.3 Shared Memory

由于 shared memory 位于芯片上，其带宽远高于 local memory 和 global memory，延迟也更低——前提是线程之间没有 bank conflict，详见下一节。

10.2.3.1 Shared Memory 与 Memory Banks

为了实现并发访问的高内存带宽，shared memory 被划分为大小相等的内存模块（banks），这些模块可以同时访问。因此，跨越 n 个不同 memory banks 的 n 个地址的内存加载或存储可以同时完成，产生的有效带宽是单个 bank 带宽的 n 倍。

然而，如果内存请求的多个地址映射到同一个 memory bank，访问将被串行化。硬件将存在 bank conflict 的内存请求拆分为所需数量的单独无冲突请求，导致有效带宽按单独内存请求的数量成比例下降。唯一的例外是当 warp 中的多个线程访问相同的 shared memory 位置时，会产生广播。在这种情况下，来自不同 banks 的多次广播被合并为从请求的 shared memory 位置到线程的单次多播。

为了最小化 bank conflict，了解内存地址如何映射到 memory banks 以及如何优化调度内存请求非常重要。

在计算能力 5.x 或更新的设备上，每个 bank 在每个时钟周期具有 32 位的带宽，连续的 32 位字被分配到连续的 banks。warp 大小为 32 个线程，bank 数量也为 32，因此 warp 中任意线程之间都可能发生 bank conflict。更多详情请参见计算能力 5.x。

10.2.3.2 Shared Memory 在矩阵乘法中的应用（C=AB）

Shared memory 支持块内线程之间的协作。当一个块中的多个线程使用来自 global memory 的相同数据时，可以使用 shared memory 仅从 global memory 访问一次该数据。Shared memory 还可用于避免非合并的内存访问，方法是以合并模式从 global memory 加载和存储数据，然后在 shared memory 中重新排序。除了 memory bank conflict 之外，warp 在 shared memory 中的非顺序或非对齐访问没有性能惩罚。

通过矩阵乘法 C = AB 的简单示例来说明 shared memory 的使用，其中 A 的维度为 M×w，B 的维度为 w×N，C 的维度为 M×N。为了保持 kernel 简洁，M 和 N 取 32 的倍数，因为当前设备的 warp 大小（w）为 32。

一种自然的分解方式是使用 w×w 个线程的块和分块大小。因此，在 w×w 分块的意义上，A 是列矩阵，B 是行矩阵，C 是它们的外积；见图 11。启动一个 N/w × M/w 的 grid，每个线程块根据 A 的一个分块和 B 的一个分块计算 C 中不同分块的元素。

Figure 11: 块列矩阵乘以块行矩阵。块列矩阵（A）乘以块行矩阵（B），得到结果乘积矩阵（C）。

为此，simpleMultiply kernel（未优化的矩阵乘法）计算矩阵 C 一个分块的输出元素。
Unoptimized matrix multiplication：

代码块

```
__global__ void simpleMultiply(float *a, float* b, float *c,
                               int N)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    float sum = 0.0f;
    for (int i = 0; i < TILE_DIM; i++) {
        sum += a[row*TILE_DIM+i] * b[i*N+col];
    }
    c[row*N+col] = sum;
}
```

在上述代码中，a、b 和 c 分别是指向矩阵 A、B 和 C 在 global memory 中的指针；blockDim.x、blockDim.y 和 TILE_DIM 都等于 w。w×w 个线程的块中的每个线程计算 C 的一个分块中的一个元素。row 和 col 分别是特定线程正在计算的 C 中元素的行和列。对 i 的 for 循环将 A 的一行乘以 B 的一列，然后将结果写入 C。

该 kernel 在 NVIDIA Tesla V100 上的有效带宽为 119.9 GB/s。为了分析性能，必须考虑 warp 在 for 循环中如何访问 global memory。每个 warp 的线程计算 C 的一个分块的一行，这依赖于 A 的一行和 B

的整个分块，如图 12 所示。

Figure 12: 计算一个分块的一行。使用 A 的一行和 B 的整个分块计算 C 中一个分块的一行。

对于 for 循环的每次迭代 i，warp 中的线程读取 B 分块的一行，这在所有计算能力下都是顺序且合并的访问。

然而，对于每次迭代 i，warp 中的所有线程从 global memory 读取矩阵 A 的相同值，因为索引 row*TILE_DIM+i 在 warp 内是常量。尽管在计算能力 2.0 或更高的设备上这种访问只需要 1 次事务，但事务中存在带宽浪费，因为 32 字节缓存段中的 8 个字中只有 1 个 4 字节字被使用。我们可以在循环的后续迭代中重用这个缓存行，最终会利用全部 8 个字；但是，当许多 warp 同时在同一个 multiprocessor 上执行时（通常情况下），缓存行可能很容易在迭代 i 和 i+1 之间从缓存中被逐出。

通过将 A 的一个分块读入 shared memory 可以提升任何计算能力设备上的性能，如下面的代码所示。Using shared memory to improve the global memory load efficiency in matrix multiplication：

代码块

```
__global__ void coalescedMultiply(float *a, float* b, float *c,
                                  int N)
{
    __shared__ float aTile[TILE_DIM][TILE_DIM];

    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    float sum = 0.0f;
    aTile[threadIdx.y][threadIdx.x] = a[row*TILE_DIM+threadIdx.x];
    __syncwarp();
    for (int i = 0; i < TILE_DIM; i++) {
        sum += aTile[threadIdx.y][i]* b[i*N+col];
    }
    c[row*N+col] = sum;
}
```

在上述代码中，A 分块中的每个元素仅从 global memory 读取一次，以完全合并的方式（无带宽浪费）加载到 shared memory。在 for 循环的每次迭代中，shared memory 中的值被广播到 warp 中的所有线程。在将 A 分块读入 shared memory 之后，使用 __syncwarp() 而不是 __syncthreads() 同步屏障调用就足够了，因为只有将数据写入 shared memory 的 warp 内的线程才会读取这些数据。该 kernel 在 NVIDIA Tesla V100 上的有效带宽为 144.4 GB/s。这说明了当硬件 L1 缓存逐出策略与应用需求不匹配，或者 L1 缓存不用于从 global memory 读取时，可以将 shared memory 用作用户管理的缓存。

可以进一步改进处理矩阵 B 的方式。在计算矩阵 C 某个分块的每一行时，整个 B 分块都会被读取。可以通过将 B 分块一次性读入 shared memory 来消除对 B 分块的重复读取（如下面的代码所示）。Improvement by reading additional data into shared memory：

```
代码块__global__ void sharedABMultiply(float *a, float* b, float *c,
                                       int N)
{
    __shared__ float aTile[TILE_DIM][TILE_DIM],
                     bTile[TILE_DIM][TILE_DIM];
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    float sum = 0.0f;
    aTile[threadIdx.y][threadIdx.x] = a[row*TILE_DIM+threadIdx.x];
    bTile[threadIdx.y][threadIdx.x] = b[threadIdx.y*N+col];
    __syncthreads();
    for (int i = 0; i < TILE_DIM; i++) {
        sum += aTile[threadIdx.y][i]* bTile[i][threadIdx.x];
    }
    c[row*N+col] = sum;
}
```

请注意，在上述代码中，读取 B 分块后需要调用 __syncthreads()，因为一个 warp 会读取由不同 warp 写入 shared memory 的数据。该程序在 NVIDIA Tesla V100 上的有效带宽为 195.5 GB/s。注意，性能提升并非由于合并访问的改善，而是由于避免了从 global memory 的冗余传输。

Table 2: 优化 C = AB 矩阵乘法的性能提升

优化措施	NVIDIA Tesla V100
无优化	119.9 GB/s
使用 shared memory 合并存储 A 分块	144.4 GB/s
使用 shared memory 消除 B 分块的冗余读取	195.5 GB/s

注意
中优先级：使用 shared memory 避免从 global memory 的冗余传输。

10.2.3.3 Shared Memory 在矩阵乘法中的应用（C=AAT）

前述矩阵乘法的一个变体可用于说明如何处理 global memory 的跨步访问以及 shared memory 的 bank conflict。该变体简单地用 A 的转置代替 B，即 C = AAT。

C = AAT 的简单实现在下面的代码中给出。
Unoptimized handling of strided accesses to global memory：

```
代码块
__global__ void simpleMultiply(float *a, float *c, int M)
{
```

```
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    float sum = 0.0f;
    for (int i = 0; i < TILE_DIM; i++) {
        sum += a[row*TILE_DIM+i] * a[col*TILE_DIM+i];
    }
    c[row*M+col] = sum;
}
```

在上面的示例中，C 的第 row 行、第 col 列元素通过计算 A 的第 row 行和第 col 行的点积得到。该 kernel 在 NVIDIA Tesla V100 上的有效带宽为 12.8 GB/s。这些结果远低于 C = AB kernel 的相应测量值。差异在于对于每次迭代 i，warp 中的线程如何访问第二项 a[col*TILE_DIM+i] 中的 A 元素。对于一个 warp 的线程，col 表示 A 转置的顺序列，因此 col*TILE_DIM 表示以 w 为跨步的 global memory 跨步访问，导致大量带宽浪费。

避免跨步访问的方法是像之前一样使用 shared memory，但此时一个 warp 将 A 的一行读入 shared memory 分块的一列，如下面的代码所示。
An optimized handling of strided accesses using coalesced reads from global memory：

代码块

```
__global__ void coalescedMultiply(float *a, float *c, int M)
{
    __shared__ float aTile[TILE_DIM][TILE_DIM],
                     transposedTile[TILE_DIM][TILE_DIM];
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    float sum = 0.0f;
    aTile[threadIdx.y][threadIdx.x] = a[row*TILE_DIM+threadIdx.x];
    transposedTile[threadIdx.x][threadIdx.y] =
        a[(blockIdx.x*blockDim.x + threadIdx.y)*TILE_DIM +
        threadIdx.x];
    __syncthreads();
    for (int i = 0; i < TILE_DIM; i++) {
        sum += aTile[threadIdx.y][i]* transposedTile[i][threadIdx.x];
    }
    c[row*M+col] = sum;
}
```

上述代码使用共享的 transposedTile 来避免点积第二项中的非合并访问，并使用前一个示例中的共享 aTile 技术来避免第一项中的非合并访问。该 kernel 在 NVIDIA Tesla V100 上的有效带宽为 140.2 GB/s。这些结果低于 C = AB 最终 kernel 所获得的结果。差异的原因在于 shared memory bank conflict。

for 循环内 transposedTile 中元素的读取是无冲突的，因为每个 warp 的线程按行读取分块，从而对 banks 产生单位跨步。然而，在将分块从 global memory 复制到 shared memory 时会发生 bank conflict。为了使从 global memory 的加载合并，数据从 global memory 顺序读取。然而，这需要按列写入 shared memory，并且由于 shared memory 中使用 w×w 分块，这导致线程之间的跨步为 w 个 banks——warp 中的每个线程都命中同一个 bank（回想 w 取 32）。这种多路 bank conflict 代价非常高。简单的解决方法是填充 shared memory 数组，使其多出一列，如下面的代码行所示。

代码块

```
__shared__ float transposedTile[TILE_DIM][TILE_DIM+1];
```

这种填充完全消除了冲突，因为现在线程之间的跨步是 w+1 个 banks（即当前设备上为 33），由于计算 bank 索引使用了模运算，这等效于单位跨步。经过此更改后，在 NVIDIA Tesla V100 上的有效带宽为 199.4 GB/s，与上一个 C = AB kernel 的结果相当。

这些优化的结果总结在表 3 中。

Table 3: 优化 C = AAT 矩阵乘法的性能提升

优化措施	NVIDIA Tesla V100
无优化	12.8 GB/s
使用 shared memory 合并 global 读取	140.2 GB/s
消除 bank conflicts	199.4 GB/s

这些结果应与表 2 中的结果进行比较。从这些表格可以看出，合理使用 shared memory 可以显著提升性能。

本节中的示例说明了使用 shared memory 的三个原因：

- 实现 global memory 的合并访问，特别是避免大步跨（对于一般矩阵，跨步远大于 32）
- 消除（或减少）从 global memory 的冗余加载
- 避免浪费的带宽

10.2.3.4 从 Global Memory 到 Shared Memory 的异步复制

CUDA 11.0 引入了异步复制特性，可在设备代码内用于显式管理从 global memory 到 shared memory 的数据异步复制。此功能使 CUDA kernel 能够将数据从 global 复制到 shared memory 的操作与计算重叠。它还避免了传统上 global memory 读取与 shared memory 写入之间存在的中间寄存器文件传输。

更多详情请参考《CUDA C++ Programming Guide》中的 memcpy_async 部分。

为了理解从 global memory 到 shared memory 的同步复制和异步复制的性能差异，考虑以下微基准测试 CUDA kernel，分别展示了同步和异步方法。异步复制在 NVIDIA A100 GPU 上具有硬件加速。

代码块

```
template <typename T>
__global__ void pipeline_kernel_sync(T *global, uint64_t *clock, size_t copy_count) {
  extern __shared__ char s[];
  T *shared = reinterpret_cast<T *>(s);

  uint64_t clock_start = clock64();

  for (size_t i = 0; i < copy_count; ++i) {
    shared[blockDim.x * i + threadIdx.x] = global[blockDim.x * i + threadIdx.x];
  }

  uint64_t clock_end = clock64();

  atomicAdd(reinterpret_cast<unsigned long long *>(clock),
            clock_end - clock_start);
}

template <typename T>
__global__ void pipeline_kernel_async(T *global, uint64_t *clock, size_t copy_count) {
  extern __shared__ char s[];
  T *shared = reinterpret_cast<T *>(s);

  uint64_t clock_start = clock64();

  //pipeline pipe;
  for (size_t i = 0; i < copy_count; ++i) {
    __pipeline_memcpy_async(&shared[blockDim.x * i + threadIdx.x],
                            &global[blockDim.x * i + threadIdx.x], sizeof(T));
  }
  __pipeline_commit();
  __pipeline_wait_prior(0);

  uint64_t clock_end = clock64();

  atomicAdd(reinterpret_cast<unsigned long long *>(clock),
            clock_end - clock_start);
}
```

同步版本 kernel 将元素从 global memory 加载到中间寄存器，然后将中间寄存器值存储到 shared memory。在异步版本 kernel 中，一旦调用 __pipeline_memcpy_async() 函数，就会发出从 global memory 加载并直接存储到 shared memory 的指令。__pipeline_wait_prior(0) 将等待 pipe 对象中的所有指令执行完毕。使用异步复制不使用任何中间寄存器。不使用中间寄存器有助于减少寄存器压力，并可以提高 kernel 占用率。使用异步复制指令从 global memory 复制到 shared memory 的数据可以缓存在 L1 cache 中，也可以选择绕过 L1 cache。如果单个 CUDA 线程复制 16 字节的元素，则可以绕过 L1 cache。这一差异如图 13 所示。

Figure 13: 同步与异步从 Global Memory 复制到 Shared Memory 的对比

我们使用每个线程 4B、8B 和 16B 的元素大小（即使用 int、int2 和 int4 作为模板参数）来评估两个 kernel 的性能。我们调整 kernel 中的 copy_count，使每个线程块复制从 512 字节到 48 MB 的数据。kernel 的性能如图 14 所示。

Figure 14: 同步与异步从 Global Memory 复制到 Shared Memory 的性能对比

从性能图表中，可以得出本实验的以下观察结果：

- 对于所有三种元素大小，当 copy_count 参数为 4 的倍数时，同步复制实现最佳性能。编译器可以优化 4 个为一组的加载和存储指令，这从锯齿形曲线可以明显看出。
- 异步复制在几乎所有情况下都实现了更好的性能。
- 异步复制不需要 copy_count 参数为 4 的倍数即可通过编译器优化最大化性能。
- 总体而言，使用 8 或 16 字节元素的异步复制可实现最佳性能。

10.2.4 Local Memory

Local memory 之所以如此命名，是因为其作用域是线程局部（local）的，而非因为其物理位置。实际上，local memory 是片外存储。因此，访问 local memory 与访问 global memory 一样昂贵。换句话说，名称中的 local 一词并不意味着更快的访问速度。

Local memory 仅用于存放自动变量。这是由 nvcc 编译器在确定寄存器空间不足以容纳变量时完成的。可能被放置到 local memory 中的自动变量包括：占用过多寄存器空间的大型结构体或数组，以及编译器确定可能被动态索引的数组。

检查 PTX 汇编代码（通过向 nvcc 传递 -ptx 或 -keep 命令行选项获得）可以发现在第一遍编译阶段变量是否已被放置到 local memory。如果是，它将使用 .local 助记符声明，并使用 ld.local 和 st.local 助记符访问。如果不是，后续编译阶段仍可能做出不同决定，如果它们发现该变量占用了目标架构过多的寄存器空间。无法针对特定变量检查这一点，但编译器在使用 --ptxas-options=-v 选项运行时会报告每个 kernel 的总 local memory 使用量（lmem）。

10.2.5 Texture Memory

只读的 texture memory 空间具有缓存。因此，纹理读取仅在缓存未命中时产生一次设备内存读取；否则，仅需从纹理缓存中读取一次。纹理缓存针对 2D 空间局部性进行了优化，因此同一 warp 中读取

接近纹理地址的线程将实现最佳性能。Texture memory 也被设计用于具有恒定延迟的流式读取；也就是说，缓存命中减少了 DRAM 带宽需求，但不减少读取延迟。

在某些寻址情况下，通过纹理读取来读取设备内存，相较于从 global 或 constant memory 读取设备内存，可能是一种有利的替代方案。

10.2.5.1 额外的纹理功能

如果使用 tex1D()、tex2D() 或 tex3D() 而不是 tex1Dfetch() 获取纹理，硬件还提供了其他功能，这些功能可能对某些应用（如图像处理）有用，如表 4 所示。

Table 4: tex1D()、tex2D() 和 tex3D() 读取的有用特性

特性	用途	注意事项
滤波（Filtering）	纹素之间的快速低精度插值	仅在纹理引用返回浮点数据时有效
归一化纹理坐标（Normalized texture coordinates）	与分辨率无关的编码	无
寻址模式（Addressing modes）	自动处理边界情况[1]	只能与归一化纹理坐标一起使用

[1] 表 4 底部行中的边界情况自动处理是指当纹理坐标落在有效寻址范围之外时如何解析。有两种选项：clamp 和 wrap。如果 x 是坐标，N 是一维纹理的纹素数量，则在 clamp 模式下，如果 x < 0，x 替换为 0，如果 1 < x，x 替换为 1-1/N。在 wrap 模式下，x 替换为 frac(x)，其中 frac(x) = x - floor(x)。floor 返回小于或等于 x 的最大整数。因此，在 N = 1 的 clamp 模式下，1.3 的 x 被 clamp 为 1.0；而在 wrap 模式下，它被转换为 0.3。

在 kernel 调用内，纹理缓存不会与 global memory 写入保持一致性，因此从同一 kernel 调用中已通过 global 存储写入的地址进行纹理读取会返回未定义数据。也就是说，如果内存位置已由先前的 kernel 调用或内存复制更新，线程可以通过纹理安全地读取该位置；但如果已由同一线程或同一 kernel 调用中的另一个线程更新，则不能安全读取。

10.2.6 Constant Memory

设备上总共有 64 KB 的 constant memory。Constant memory 空间具有缓存。因此，从 constant memory 读取仅在缓存未命中时产生一次设备内存读取；否则，仅需从常量缓存中读取一次。Warp 中线程对不同地址的访问会被串行化，因此代价随 warp 中所有线程读取的唯一地址数量线性增长。因此，当同一 warp 中的线程仅访问少数几个不重复的位置时，常量缓存效果最佳。如果 warp 中的所有线程访问同一位置，则 constant memory 可以像寄存器访问一样快。

10.2.7 Registers

通常，访问寄存器每条指令消耗零额外时钟周期，但由于寄存器读写依赖和寄存器内存 bank conflict 可能导致延迟。

编译器和硬件线程调度器将尽可能优化指令调度，以避免寄存器内存 bank conflict。应用程序无法直接控制这些 bank conflict。特别是，没有与寄存器相关的原因需要将数据打包成向量数据类型，如 float4 或 int4 类型。

10.2.7.1 寄存器压力

当可用寄存器不足以完成给定任务时，就会发生寄存器压力。尽管每个 multiprocessor 包含数千个 32 位寄存器（参见《CUDA C++ Programming Guide》的功能和技术规格），但这些寄存器需在并发线程之间分配。为防止编译器分配过多寄存器，可以使用 -maxrregcount=N 编译器命令行选项或 launch bounds kernel 定义限定符（参见《CUDA C++ Programming Guide》的执行配置）来控制每个线程分配的最大寄存器数量。

10.3 内存分配

通过 cudaMalloc() 和 cudaFree() 进行设备内存分配和释放是开销昂贵的操作。建议使用 cudaMallocAsync() 和 cudaFreeAsync()，它们是流排序的池分配器，用于管理设备内存。

10.4 NUMA 最佳实践

一些较新的 Linux 发行版默认启用自动 NUMA 平衡（或"AutoNUMA"）。在某些情况下，自动 NUMA 平衡执行的操作可能会降低在 NVIDIA GPU 上运行的应用程序的性能。为获得最佳性能，用户应手动调整其应用程序的 NUMA 特性。

最佳 NUMA 调优取决于每个应用程序和节点的特性及所需的硬件亲和性，但通常在 NVIDIA GPU 上进行计算的应用程序建议选择禁用自动 NUMA 平衡的策略。例如，在 IBM Newell POWER9 节点上（CPU 对应 NUMA 节点 0 和 8），使用：

代码块

```
numactl --membind=0,8
```

将内存分配绑定到 CPU。

---

11. 执行配置优化

保持设备上 multiprocessors 尽可能繁忙是实现良好性能的关键之一。如果工作在 multiprocessors 之间的分配不均衡，设备将提供次优性能。因此，设计应用程序时应以最大化硬件利用率的方式使用线程和块，并限制阻碍工作自由分配的做法。这一努力中的关键概念是占用率（occupancy），将在以下各节中解释。

在某些情况下，通过设计应用程序使多个独立的 kernel 能够同时执行，也可以提高硬件利用率。多个 kernel 同时执行称为并发 kernel 执行，将在下文描述。

另一个重要概念是为特定任务分配的系统资源管理。如何管理这种资源利用率将在本章最后几节中讨论。

11.1 占用率

在 CUDA 中，线程指令按顺序执行，因此，当一个 warp 暂停或停滞时，执行其他 warp 是隐藏延迟并保持硬件繁忙的唯一方法。因此，与 multiprocessor 上活跃 warp 数量相关的某些指标对于确定硬件保持繁忙的有效性非常重要。这个指标就是占用率（occupancy）。

占用率是每个 multiprocessor 上活跃 warp 数量与最大可能活跃 warp 数量的比率。（要确定后者，请参阅 deviceQuery CUDA 示例或参考计算能力。）另一种看待占用率的方式是硬件处理 warp 能力中实际在使用的百分比。

更高的占用率并不总是等同于更高的性能——在超过某一点后，额外的占用率不会改善性能。然而，低占用率始终会干扰隐藏内存延迟的能力，导致性能下降。

CUDA kernel 所需的每线程资源可能以不希望的方式限制最大块大小。为了保持对未来硬件和工具包的前向兼容性，并确保至少一个线程块可以在 SM 上运行，开发者应该包含单个参数的 `__launch_bounds__(maxThreadsPerBlock)`，该参数指定 kernel 将启动的最大块大小。不这样做可能导致"请求的资源过多无法启动"错误。提供两个参数版本的 `__launch_bounds__(maxThreadsPerBlock,minBlocksPerMultiprocessor)` 在某些情况下可以提高性能。minBlocksPerMultiprocessor 的正确值应通过对每个 kernel 的详细分析来确定。

11.1.1 计算占用率

决定占用率的几个因素之一是寄存器可用性。寄存器存储使线程能够将局部变量保持在附近，以实现低延迟访问。然而，寄存器集合（称为寄存器文件）是所有驻留在 multiprocessor 上的线程必须共享的有限资源。寄存器是一次性分配给整个块的。因此，如果每个线程块使用了大量寄存器，可以驻留在 multiprocessor 上的线程块数量就会减少，从而降低 multiprocessor 的占用率。每个线程的最大寄存器数量可以在编译时通过 -maxrregcount 选项按文件手动设置，或通过 `__launch_bounds__` 限定符按 kernel 设置（参见寄存器压力）。

为了计算占用率，每个线程使用的寄存器数量是关键因素之一。例如，在 CUDA 计算能力 7.0 的设备上，每个 multiprocessor 有 65,536 个 32 位寄存器，最多可以有 2048 个并发线程驻留（64 warps × 32 线程/ warp）。这意味着在这些设备中，要使 multiprocessor 达到 100% 占用率，每个线程最多可以使用 32 个寄存器。然而，这种确定寄存器数量如何影响占用率的方法没有考虑寄存器分配粒度。例如，在计算能力 7.0 的设备上，使用每线程 37 个寄存器、128 线程块的 kernel 在 12 个活跃 128 线程块/ multiprocessor 的情况下可实现 75% 的占用率；而使用相同每线程 37 个寄存器、320 线程块的 kernel 则只有 63% 的占用率，因为只有四个 320 线程块可以驻留在一个 multiprocessor 上。此外，寄存器分配会向上取整到每 warp 最近的 256 个寄存器。

可用寄存器数量、每个 multiprocessor 上最大并发驻留线程数以及寄存器分配粒度因不同计算能力而异。由于寄存器分配中的这些细微差别，以及 multiprocessor 的 shared memory 也在驻留线程块之间分配，因此寄存器使用与占用率之间的确切关系可能难以确定。nvcc 的 --ptxas-options=-v 选项可

详细列出每个 kernel 使用的每线程寄存器数量。参见《CUDA C++ Programming Guide》的硬件多线程部分了解各种计算能力设备的寄存器分配公式，以及功能和技术规格部分了解这些设备上可用的寄存器总数。另外，NVIDIA 在 Nsight Compute 中提供了占用率计算器；参考 https://docs.nvidia.com/nsight-compute/NsightCompute/index.html#occupancy-calculator。

Figure 15: 使用 CUDA 占用率计算器预测 GPU multiprocessor 占用率

应用程序还可以使用 CUDA Runtime 中的 Occupancy API（例如 cudaOccupancyMaxActiveBlocksPerMultiprocessor）根据运行时参数动态选择启动配置。

11.2 隐藏寄存器依赖

注意
中优先级：为隐藏寄存器依赖引起的延迟，在 multiprocessor 上保持足够数量的活跃线程（即保持足够的占用率）。

寄存器依赖发生在一个指令使用了先前指令写入的寄存器中的结果时。大多数算术指令的延迟在计算能力 7.0 的设备上通常为 4 个周期。因此线程在使用算术结果之前必须等待大约 4 个周期。然而，这种延迟可以通过其他 warp 中线程的执行来完全隐藏。详见 Registers。

11.3 线程和块启发式规则

注意
中优先级：每个块的线程数应为 32 个线程的倍数，因为这样可以提供最佳计算效率并有助于合并访问。

grid 中块的维度和大小以及每个块中线程的维度和大小都是重要因素。这些参数的多维特性使得多维问题更容易映射到 CUDA，但在性能方面不起作用。因此，本节讨论大小而非维度。

延迟隐藏和占用率取决于每个 multiprocessor 上的活跃 warp 数量，这由执行参数以及资源（寄存器和 shared memory）约束隐式决定。选择执行参数是在延迟隐藏（占用率）和资源利用率之间取得平衡。

选择执行配置参数应协同进行；然而，每个参数各自有一些启发式规则。在选择第一个执行配置参数——即每个 grid 的块数，或 grid 大小——时，首要考虑的是保持整个 GPU 繁忙。grid 中的块数应大于 multiprocessor 的数量，以便所有 multiprocessor 至少有 1 个块可执行。此外，每个 multiprocessor 上应有多个活跃块，以便那些不在等待 __syncthreads() 的块可以保持硬件繁忙。此建议受资源可用性约束，因此应在第二个执行参数——每个块的线程数，或块大小——以及 shared memory 使用量的上下文中确定。为了扩展到未来的设备，每次 kernel 启动的块数应为数千个。

在选择块大小时，重要的是要记住多个并发块可以驻留在一个 multiprocessor 上，因此占用率不仅仅由块大小决定。特别是，更大的块大小并不意味着更高的占用率。

如占用率部分所述，更高的占用率并不总是等同于更好的性能。例如，将占用率从 66% 提升到 100% 通常不会转化为类似的性能增长。低占用率 kernel 比较高高占用率 kernel 拥有更多的每线程可用寄存器，这可能导致更少的寄存器溢出到 local memory；特别是，在具有高度暴露的指令级并行性（ILP）的情况下，有时可以用低占用率完全覆盖延迟。

选择块大小时涉及许多这样的因素，不可避免地需要进行一些实验。然而，应遵循一些经验法则：

- 每个块的线程数应为 warp 大小的倍数，以避免在未满 warp 上浪费计算并促进合并访问。
- 每个块至少应使用 64 个线程，且仅在每个 multiprocessor 上有多个并发块时才有效。
- 128 到 256 个线程的块大小是对不同块大小进行实验的良好初始范围。
- 如果延迟影响性能，应使用多个较小的线程块而不是一个大的线程块。这对频繁调用 __syncthreads() 的 kernel 特别有益。

请注意，当线程块分配的寄存器超过 multiprocessor 上可用的数量时，kernel 启动将失败；同样，当请求过多的 shared memory 或线程时也会失败。

11.4 Shared Memory 的影响

Shared memory 在多种情况下都有帮助，例如帮助合并访问或消除对 global memory 的冗余访问。然而，它也可能成为占用率的约束。在许多情况下，kernel 所需的 shared memory 量与所选的块大小相关，但线程到 shared memory 元素的映射不必是一对一的。例如，可能希望在 kernel 中使用 64×64 元素的 shared memory 数组，但由于每个块的最大线程数为 1024，无法启动每个块 64×64 线程的 kernel。在这种情况下，可以启动 32×32 或 64×16 线程的 kernel，每个线程处理 shared memory 数组的四个元素。即使在线程数限制（如每块最大线程数）不是问题的情况下，使用单个线程处理 shared memory 数组的多个元素可能也是有益的。这是因为某些对每个元素通用的操作可以由线程一次性执行，将开销分摊到线程处理的 shared memory 元素数量上。

确定性能对占用率敏感性的一种有用技术是，通过实验改变动态分配的 shared memory 数量（在执行配置的第三个参数中指定）。通过简单地增加此参数（不修改 kernel），可以有效降低 kernel 的占用率并测量其对性能的影响。

11.5 并发 Kernel 执行

正如异步传输与计算重叠部分所述，CUDA stream 可用于将 kernel 执行与数据传输重叠。在支持并发 kernel 执行的设备上，stream 还可以用于同时执行多个 kernel，以更充分地利用设备的 multiprocessors。设备是否具有此能力由 cudaDeviceProp 结构的 concurrentKernels 字段指示（或列在 deviceQuery CUDA 示例的输出中）。并发执行需要非默认 stream（stream 0 之外的 stream），因为使用默认 stream 的 kernel 调用只有在设备上所有先前调用（任何 stream 中）完成后才开始，并且设备上的任何操作（任何 stream 中）在它们完成之前都不会开始。

以下示例说明了基本技术。因为 kernel1 和 kernel2 在不同的非默认 stream 中执行，一个能够的设备可以同时执行这些 kernel。

代码块

```
1   cudaStreamCreate(&stream1);
2   cudaStreamCreate(&stream2);
3   kernel1<<<grid, block, 0, stream1>>>(data_1);
4   kernel2<<<grid, block, 0, stream2>>>(data_2);
```

11.6 多上下文

CUDA 工作在特定 GPU 的进程空间内进行，该空间称为上下文（context）。上下文封装了该 GPU 的 kernel 启动和内存分配，以及支持构造（如页表）。上下文在 CUDA Driver API 中是显式的，但在 CUDA Runtime API 中是完全隐式的，后者会自动创建和管理上下文。

在 CUDA Driver API 中，一个 CUDA 应用程序进程可能为给定的 GPU 创建多个上下文。如果多个 CUDA 应用程序进程同时访问同一个 GPU，这几乎总是意味着多个上下文，因为上下文与特定主机进程绑定，除非使用了 Multi-Process Service。

虽然可以在给定 GPU 上同时分配多个上下文（及其相关资源，如 global memory 分配），但在任何给定时刻，该 GPU 上只能有一个上下文执行工作；共享同一 GPU 的上下文是时间分片的。创建额外的上下文会带来每上下文数据的内存开销和上下文切换的时间开销。此外，上下文切换的需求可能在来自多个上下文的工作原本可以并发执行时降低利用率（另见并发 Kernel 执行）。

因此，在同一 CUDA 应用程序中最好避免每个 GPU 使用多个上下文。为此，CUDA Driver API 提供了访问和管理每个 GPU 上称为"主上下文"（primary context）的特殊上下文的方法。这与 CUDA Runtime 在线程还没有当前上下文时隐式使用的上下文相同。

代码块

```
// When initializing the program/library
CUcontext ctx;
cuDevicePrimaryCtxRetain(&ctx, dev);

// When the program/library launches work
cuCtxPushCurrent(ctx);
kernel<<<...>>>(...);
cuCtxPopCurrent(&ctx);

// When the program/library is finished with the context
cuDevicePrimaryCtxRelease(dev);
```

注意
NVIDIA-SMI 可用于将 GPU 配置为独占进程模式，这将每个 GPU 的上下文数量限制为 1 个。此上下文可以在创建进程内被任意数量的线程设为当前，如果设备上已存在使用 CUDA driver API 创建的非主上下文，则 cuDevicePrimaryCtxRetain 将失败。

---

12. 指令优化

了解指令的执行方式通常可以进行有用的低级优化，特别是在频繁运行的代码（即程序中的所谓热点）中。最佳实践建议在所有更高级别的优化完成之后才进行此优化。

12.1 算术指令

要提高指令吞吐量，应用程序应：

- 最小化使用低吞吐量算术指令；这包括在不影响最终结果的情况下用精度换取速度，例如使用 intrinsic 函数而非常规函数（intrinsic 函数列在《CUDA C++ Programming Guide》中），使用单精度而非双精度，或将非正规数刷新为零；
- 最小化由控制流指令引起的 warp 发散，详见控制流指令部分；
- 减少指令数量，例如，通过优化掉同步点（如同步指令部分所述）或使用受限指针（如《CUDA C++ Programming Guide》所述）。

在本节中，吞吐量以每个 multiprocessor 每个时钟周期的操作数给出。对于大小为 32 的 warp，一条指令对应 32 个操作，因此如果 N 是每个时钟周期的操作数，则指令吞吐量为每个时钟周期 N/32 条指令。

所有吞吐量均针对一个 multiprocessor。必须乘以设备中 multiprocessor 的数量才能得到整个设备的吞吐量。

12.1.1 原生算术指令的吞吐量

下表给出了各种计算能力设备上硬件原生支持的算术指令的吞吐量。

注意
此表反映了所述操作的最大理论吞吐量。在某些情况下，这些吞吐量可能只能通过特定的指令序列实现，这在使用编译器时需要特别注意。大多数行提供了一个 PTX 指令作为预期能达到所列吞吐量的低级指令示例。此 PTX 指令可能不是能执行所述操作的唯一指令。

Table 5: 原生算术指令的吞吐量（每个 Multiprocessor 每个时钟周期的结果数）

计算能力	7.5	8.0	8.6	8.9	9.0	10.0	12.0
16 位浮点加、乘、乘加（2 路 SIMD）：add.f16x2	64[3]	128[4]	64	128	64		
32 位浮点加、乘、乘加：add.f32	64					128	
64 位浮点加、乘、乘加：add.f64	2	32	2	64	64	2	
	16[5]						

32 位近似浮点倒数、平方根倒数、以 2 为底的对数、以 2 为底的指数、正弦、余弦：lg2.approx.f32							
32 位整数加、减：add.s32						128	
32 位整数扩展精度加、扩展精度减、三操作数加法：多条 PTX 指令						64	
32 位整数乘、乘加、扩展精度乘加：mad.lo.s32	64[6]						
32 位整数移位：shl.b32	64						
64 位整数加：add.s64	32	64					
32 位整数比较、最小值、最大值：min.s32	64					128	
32 位整数字节反转：not.b32	64						

32 位按位 AND、OR、XOR：xor.b32	64						
前导零计数、最高非符号位：clz.b32	16						
种群计数：popc.b32	16						
warp shuffle[7]：shfl.sync.idx.b32	16		32				
warp vote[8]：vote.sync.ballot.b32	64					128	
绝对差之和：sad.s32	64	32					
2 路 SIMD 绝对差：多条 PTX 指令		多条指令					
4 路 SIMD 绝对差：vabsdiff4.u32.u32.u32	64[9]	32[9]					
从 8 位和 16 位整数到 32 位整数类型的类型转换：cvt.u32.u16	64						

从和到 64 位浮点类型的类型转换：cvt.f64.f32	2	16	2	2	16	2	
16 位和 32 位浮点类型之间的类型转换：cvt.f32.f16	64[10]						
32 位整数和 32 位浮点类型之间的类型转换：cvt.rn.f32.s32	16					64[11]	
DPX（32 位和 2 路 SIMD 16 位）：多条 PTX 指令		多条指令	64[12]				64[13]

[3] __nv_bfloat16 使用多条指令

[4] __nv_bfloat16 为 64

[5] 倒数：使用 __fdividef(1.F, x) 近似

[6] 扩展精度为 32

[7] 假设线程已收敛

[8] 假设线程已收敛

[9] 有符号版本为 22

[10] __nv_bfloat16->float 为 32，float->__half 为 16，float->__nv_bfloat16 (SM > 7.5) 为 16，float->__nv_bfloat16 (SM 7.5) 使用多条指令

[11] float->32 位整数类型为 16

[12] 带布尔谓词输出的 intrinsic 使用多条指令

[13] 2 个操作数的 intrinsic 为 128，带布尔谓词输出的 intrinsic 使用多条指令

其他指令和函数是在原生指令之上实现的。不同计算能力设备的实现可能不同，编译后的原生指令数量可能随每个编译器版本而变化。对于复杂函数，可能有多个代码路径取决于输入。cuobjdump 可用于检查 cubin 对象中的特定实现。

某些函数的实现可在 CUDA 头文件（math_functions.h、device_functions.h 等）中直接获取。

通常，使用 -ftz=true（非正规数刷新为零）编译的代码往往比使用 -ftz=false 编译的代码具有更高的性能。类似地，使用 -prec-div=false（较低精度的除法）编译的代码往往比使用 -prec-div=true 编译的代码性能更高，而使用 -prec-sqrt=false（较低精度的平方根）编译的代码往往比使用 -prec-sqrt=true 编译的代码性能更高。nvcc 用户手册详细描述了这些编译标志。

单精度浮点除法

__fdividef(x, y)（参见《CUDA C++ Programming Guide》）提供比除法运算符更快的单精度浮点除法。

单精度浮点平方根倒数

为保持 IEEE-754 语义，编译器仅在倒数和平方根均为近似时（即使用 -prec-div=false 和 -prec-sqrt=false）才会将 1.0/sqrtf() 优化为 rsqrtf()。因此，建议在需要时直接调用 rsqrtf()。

单精度浮点平方根

单精度浮点平方根实现为平方根倒数后接一个倒数，而非平方根倒数后接一个乘法，以便对 0 和无穷大给出正确结果。

正弦和余弦

sinf(x)、cosf(x)、tanf(x)、sincosf(x) 以及相应的双精度指令开销要大得多，如果参数 x 的绝对值很大则更是如此。

更准确地说，参数约减代码（实现参见《CUDA C++ Programming Guide》）包含两个代码路径，分别称为快速路径和慢速路径。

快速路径用于绝对值足够小的参数，基本上由少量乘加操作组成。慢速路径用于绝对值大的参数，由为在整个参数范围内实现正确结果所需的冗长计算组成。

目前，三角函数的参数约减代码对于单精度函数，当参数绝对值小于 105615.0f 时选择快速路径；对于双精度函数，当参数绝对值小于 2147483648.0 时选择快速路径。

由于慢速路径需要比快速路径更多的寄存器，已尝试通过将一些中间变量存储在 local memory 中来减少慢速路径的寄存器压力，这可能会因 local memory 的高延迟和低带宽（参见《CUDA C++ Programming Guide》）而影响性能。目前，单精度函数使用 28 字节的 local memory，双精度函数使用 44 字节。然而，确切的数量可能会发生变化。

由于慢速路径中的冗长计算和 local memory 的使用，当需要慢速路径约减时，这些三角函数的吞吐量比快速路径约减低一个数量级。

整数运算

整数除法和取模运算代价高昂，因为它们编译为多达 20 条指令。在某些情况下可以用位运算替代：如果 n 是 2 的幂，(i/n) 等价于 (i>>log2(n))，(i%n) 等价于 (i&(n-1))；如果 n 是字面量，编译器将进行这

些转换。

__brev 和 __popc 映射到单条指令，__brevll 和 __popcll 映射到几条指令。

__[u]mul24 是遗留的 intrinsic 函数，已没有任何理由再使用。

半精度算术

为了实现 16 位精度浮点加法、乘法或乘加的良好性能，建议对半精度使用 half2 数据类型，对 __nv_bfloat16 精度使用 __nv_bfloat162。然后可以使用向量 intrinsic 函数（例如 __hadd2、__hsub2、__hmul2、__hfma2）在一条指令中执行两个操作。使用 half2 或 __nv_bfloat162 代替两次使用 half 或 __nv_bfloat16 的调用也可能有助于其他 intrinsic 函数（如 warp shuffle）的性能。

提供了 intrinsic 函数 __halves2half2 将两个半精度值转换为 half2 数据类型。

提供了 intrinsic 函数 __halves2bfloat162 将两个 __nv_bfloat 精度值转换为 __nv_bfloat162 数据类型。

类型转换

有时，编译器必须插入转换指令，引入额外的执行周期。这适用于以下情况：

- 对类型为 char 或 short 的变量进行操作的函数，其操作数通常需要转换为 int；
- 双精度浮点常量（即那些不带任何类型后缀定义的常量）用作单精度浮点计算的输入（按 C/C++ 标准要求）。

后一种情况可以通过使用单精度浮点常量来避免，这些常量以 f 后缀定义，如 3.141592653589793f、1.0f、0.5f。

12.1.2 控制流指令

任何控制流指令（if、switch、do、for、while）都可能通过导致同一 warp 的线程发散（即遵循不同的执行路径）来显著影响有效指令吞吐量。如果发生这种情况，不同的执行路径必须串行化，增加了该 warp 执行的指令总数。

在控制流依赖于线程 ID 的情况下，为获得最佳性能，应编写控制条件以最小化发散 warp 的数量。这是可能的，因为 warp 在块中的分布是确定性的（参见《CUDA C++ Programming Guide》）。一个简单的例子是控制条件仅依赖于 (threadIdx / warpSize)，其中 warpSize 是 warp 大小。在这种情况下，没有 warp 会发散，因为控制条件与 warp 完全对齐。

有时，编译器可能会展开循环，或者可能使用分支预测来优化掉短的 if 或 switch 块，如下文详述。在这些情况下，任何 warp 都不会发散。程序员也可以使用 `#pragma unroll` 指令来控制循环展开（参见《CUDA C++ Programming Guide》）。

使用分支预测时，执行依赖于控制条件的指令都不会被跳过。相反，每条指令都与一个每线程条件码或谓词相关联，该谓词根据控制条件设置为 true 或 false，虽然每条指令都被调度执行，但只有谓词为 true 的指令才会实际执行。谓词为 false 的指令不写入结果，也不计算地址或读取操作数。

12.1.3 同步指令

__syncthreads() 的吞吐量在计算能力 6.0 的设备上为每个时钟周期 32 个操作，在计算能力 7.x 和 8.x 的设备上为每个时钟周期 16 个操作，在计算能力 5.x、6.1 和 6.2 的设备上为每个时钟周期 64 个操作。

注意，__syncthreads() 可能会通过强制 multiprocessor 空闲而影响性能，详见《CUDA C++ Programming Guide》。

12. [续] 指令优化

12.1.2 控制流指令

任何控制流指令（`if`、`switch`、`do`、`for`、`while`）都可能显著影响有效指令吞吐量，因为它们会导致同一 warp 中的线程发生分支（即沿着不同的执行路径）。如果发生这种情况，不同的执行路径必须被序列化执行，从而增加该 warp 执行的总指令数。

在控制流依赖于线程 ID 的情况下，为获得最佳性能，应将控制条件编写成最小化分支 warp 的数量。这是可行的，因为 warp 在 block 中的分布是确定性的，如 CUDA C++ 编程指南中所述。一个简单的例子是当控制条件仅依赖于 `(threadIdx / warpSize)` 时（其中 `warpSize` 是 warp 大小）。在这种情况下，没有 warp 会发生分支，因为控制条件与 warp 完美对齐。

有时，编译器可能会展开循环，或者使用分支预测来优化掉短的 `if` 或 `switch` 块，详见下文。在这些情况下，没有任何 warp 会发生分支。程序员也可以使用 `#pragma unroll` 指令来控制循环展开（参见 CUDA C++ 编程指南）。

当使用分支预测时，任何执行依赖于控制条件的指令都不会被跳过。相反，每条指令都与一个按线程的条件码或谓词相关联，该条件码或谓词根据控制条件被设置为 true 或 false，尽管每条指令都被调度执行，但只有谓词为 true 的指令才会实际执行。谓词为 false 的指令不会写入结果，也不会计算地址或读取操作数。

12.1.3 同步指令

`__syncthreads()` 的吞吐量对于计算能力 6.0 的设备为每个时钟周期 32 次操作，对于计算能力 7.x 和 8.x 的设备为每个时钟周期 16 次操作，对于计算能力 5.x、6.1 和 6.2 的设备为每个时钟周期 64 次操作。

请注意，`__syncthreads()` 可能会通过强制多处理器空闲来影响性能，详见 CUDA C++ 编程指南。

12.1.4 除法与模运算

> 注意
> 低优先级：使用移位操作来避免昂贵的除法和模运算。

整数除法和模运算开销特别大，应尽可能避免或替换为位运算：如果 (n) 是 2 的幂，则 (i/n) 等价于 (i \gg {log2}(n))，(i% n) 等价于 (i&\left({n - 1} \right))。

如果 n 是字面量，编译器将执行这些转换。（更多信息请参考 CUDA C++ 编程指南中的性能指南。）

12.1.5 循环计数器：有符号与无符号

> 注意
> 中低优先级：使用有符号整数而非无符号整数作为循环计数器。

在 C 语言标准中，无符号整数溢出语义是明确定义的，而有符号整数溢出会导致未定义结果。因此，编译器对有符号算术可以比无符号算术进行更激进的优化。这对循环计数器尤为重要：由于循环计数器的值通常始终为正，因此很容易倾向于将计数器声明为无符号。然而，为了获得稍好的性能，应该将其声明为有符号。

例如，考虑以下代码：

代码块

```
for (i = 0; i < n; i++) {
    out[i] = in[offset + stride*i];
}
```

这里，子表达式 `stride*i` 可能溢出 32 位整数，因此如果 `i` 被声明为无符号，溢出语义会阻止编译器使用一些本来可以应用的优化，如强度削减。如果改为将 `i` 声明为有符号，其中溢出语义是未定义的，编译器就有更大的自由度来使用这些优化。

12.1.6 倒数平方根

倒数平方根应始终显式地通过 `rsqrtf()`（单精度）和 `rsqrt()`（双精度）来调用。编译器仅在不违反 IEEE-754 语义的情况下将 `1.0f/sqrtf(x)` 优化为 `rsqrtf()`。

12.1.7 其他算术指令

> 注意
> 低优先级：避免双精度自动转换为单精度。

编译器有时必须插入转换指令，这会引入额外的执行周期。以下情况属于此列：

- 操作 `char` 或 `short` 类型变量的函数，其操作数通常需要转换为 `int`
- 双精度浮点常量（未带任何类型后缀定义的常量）用作单精度浮点计算的输入

后一种情况可以通过使用带 `f` 后缀的单精度浮点常量来避免，例如 `3.141592653589793f`、`1.0f`、`0.5f`。

对于单精度代码，强烈建议使用 `float` 类型和单精度数学函数。

还应注意，CUDA 数学库的互补误差函数 `erfcf()` 特别快，且具有完全的单精度精度。

12.1.8 小分数指数的幂运算

对于某些分数指数，通过使用平方根、立方根及其逆运算，可以显著加速幂运算，相比使用 `pow()` 快得多。对于那些指数无法精确表示为浮点数的幂运算（如 1/3），这也可以提供更精确的结果，因为

使用 `pow()` 会放大初始表示误差。

下表中的公式对 x >= 0, x != -0（即 `signbit(x) == 0`）有效。

表 6 小分数幂运算公式

计算	公式
x1/9	r = rcbrt(rcbrt(x))
x-1/9	r = cbrt(rcbrt(x))
x1/6	r = rcbrt(rsqrt(x))
x-1/6	r = rcbrt(sqrt(x))
x1/4	r = rsqrt(rsqrt(x))
x-1/4	r = sqrt(rsqrt(x))
x1/3	r = cbrt(x)
x-1/3	r = rcbrt(x)
x1/2	r = sqrt(x)
x-1/2	r = rsqrt(x)
x2/3	r = cbrt(x); r = r*r
x-2/3	r = rcbrt(x); r = r*r
x3/4	r = sqrt(x); r = r*sqrt(r)
x-3/4	r = rsqrt(x); r = r*sqrt(r)
x7/6	r = x*rcbrt(rsqrt(x))
x-7/6	r = (1/x) * rcbrt(sqrt(x))
x5/4	r = x*rsqrt(rsqrt(x))
x-5/4	r = (1/x)*sqrt(rsqrt(x))
x4/3	r = x*cbrt(x)
x-4/3	r = (1/x)*rcbrt(x)
x3/2	r = x*sqrt(x)
x-3/2	r = (1/x)*rsqrt(x)

12.1.9 数学库

> 注意
> 中优先级：当速度优先于精度时，使用快速数学库。

支持两种类型的运行时数学运算。它们可以通过名称来区分：一些名称带有前置下划线，而另一些则没有（例如 `__functionName()` 与 `functionName()`）。遵循 `__functionName()` 命名约定的函数直接映射到硬件级别。它们更快，但精度稍低（例如 `__sinf(x)` 和 `__expf(x)`）。遵循 `functionName()` 命名约定的函数较慢，但精度更高（例如 `sinf(x)` 和 `expf(x)`）。`__sinf(x)`、`__cosf(x)` 和 `__expf(x)` 的吞吐量远大于 `sinf(x)`、`cosf(x)` 和 `expf(x)`。如果参数 `x` 的幅度需要规约，后者会变得更加昂贵（大约慢一个数量级）。此外，在这些情况下，参数规约代码使用局部内存，由于局部内存的高延迟，这会进一步影响性能。更多细节请参见 CUDA C++ 编程指南。

还请注意，当需要计算同一参数的正弦和余弦时，应使用 `sincos` 指令族来优化性能：

- `__sincosf()` 用于单精度快速数学（见下一段）
- `sincosf()` 用于常规单精度
- `sincos()` 用于双精度

nvcc 的 `-use_fast_math` 编译器选项将每个 `functionName()` 调用强制替换为等效的 `__functionName()` 调用。它还会禁用单精度非规范化数支持，并总体上降低单精度除法的精度。这是一种激进的优化，可能降低数值精度并改变特殊情况的处理方式。更稳健的方法是根据性能增益和可容忍的行为变化，有选择性地引入对快速内建函数的调用。请注意，此开关仅对单精度浮点有效。

> 注意
> 中优先级：在可能的情况下，优先使用更快、更专业的数学函数，而不是较慢、更通用的函数。

对于小整数次幂（例如 x2 或 x3），显式乘法几乎肯定比使用通用幂运算例程（如 `pow()`）更快。虽然编译器优化改进不断致力于缩小这一差距，但显式乘法（或使用等效的专用内联函数或宏）可以具有显著优势。当需要同一底数的多个幂时（例如在接近位置同时计算 x2 和 x5），这一优势会增大，因为这有助于编译器的公共子表达式消除（CSE）优化。

对于以 2 或 10 为底的幂运算，请使用 `exp2()` 或 `expf2()` 以及 `exp10()` 或 `expf10()` 函数，而不是 `pow()` 或 `powf()` 函数。`pow()` 和 `powf()` 都是重量级函数，在寄存器压力和指令计数方面开销很大，因为通用幂运算中存在许多特殊情况，且难以在底数和指数的整个范围内达到高精度。而 `exp2()`、`exp2f()`、`exp10()` 和 `exp10f()` 函数在性能方面与 `exp()` 和 `expf()` 相似，可以比其 `pow()` / `powf()` 等价函数快十倍。

对于指数为 1/3 的幂运算，使用 `cbrt()` 或 `cbrtf()` 函数而不是通用幂运算函数 `pow()` 或 `powf()`，因为前者明显快于后者。同样，对于指数为 -1/3 的幂运算，使用 `rcbrt()` 或 `rcbrtf()`。

将 `sin(π*<expr>)` 替换为 `sinpi(<expr>)`，将 `cos(π*<expr>)` 替换为 `cospi(<expr>)`，将 `sincos(π*<expr>)` 替换为

`sincospi(<expr>)`。这在精度和性能两方面都有优势。作为一个具体示例，要以度而不是弧度计算正弦函数，请使用 `sinpi(x/180.0)`。类似地，当函数参数形式为 `π*<expr>` 时，单精度函数 `sinpif()`、`cospif()` 和 `sincospif()` 应替换对 `sinf()`、`cosf()` 和 `sincosf()` 的调用。（`sinpi()` 相对于 `sin()` 的性能优势源于简化的参数规约；精度优势则是由于 `sinpi()` 仅隐式地乘以 π，有效地使用无限精确的数学 π，而不是单精度或双精度近似值。）

12.1.10 精度相关的编译器标志

默认情况下，nvcc 编译器生成符合 IEEE 标准的代码，但它也提供选项来生成稍不精确但更快的代码：

- `-ftz=true`（非规范化数刷新为零）
- `-prec-div=false`（精度较低的除法）
- `-prec-sqrt=false`（精度较低的平方根）

另一个更激进的选项是 `-use_fast_math`，它将每个 `functionName()` 调用强制替换为等效的 `__functionName()` 调用。这使得代码运行更快，代价是精度和准确性降低。参见数学库。

12.2 内存指令

> 注意
> 高优先级：尽量减少全局内存的使用。尽可能优先使用共享内存访问。

内存指令包括任何从共享、局部或全局内存读取或写入的指令。当访问未缓存的局部或全局内存时，存在数百个时钟周期的内存延迟。

例如，以下示例代码中的赋值运算符具有高吞吐量，但关键的是，从全局内存读取数据存在数百个时钟周期的延迟：

代码块

```
__shared__ float shared[32];
__device__ float device[32];
shared[threadIdx.x] = device[threadIdx.x];
```

如果有足够多的独立算术指令可以在等待全局内存访问完成期间发射，线程调度器可以隐藏大部分全局内存延迟。然而，最好尽可能避免访问全局内存。

13. 控制流

13.1 分支与发散

> 注意
> 高优先级：避免同一 warp 内的不同执行路径。

控制流指令（`if`、`switch`、`do`、`for`、`while`）可能会通过导致同一 warp 中的线程发散（即沿着不同执行路径）而显著影响指令吞吐量。如果发生这种情况，不同的执行路径必须分别执行；这会增加该 warp 执行的总指令数。

在控制流依赖于线程 ID 的情况下，为获得最佳性能，应将控制条件编写成最小化发散 warp 的数量。

这是可行的，因为 warp 在 block 中的分布是确定性的，如 CUDA C++ 编程指南的 SIMT 架构部分所述。一个简单的例子是当控制条件仅依赖于 `(threadIdx / WSIZE)` 时（其中 `WSIZE` 是 warp 大小）。在这种情况下，没有 warp 会发散，因为控制条件与 warp 完美对齐。

对于仅包含少量指令的分支，warp 发散通常只会导致轻微的性能损失。例如，编译器可能使用谓词化来避免实际的分支。相反，所有指令都被调度执行，但每个线程的条件码或谓词控制哪些线程执行这些指令。谓词为 false 的线程不会写入结果，也不会计算地址或读取操作数。

从 Volta 架构开始，独立线程调度允许 warp 在数据依赖的条件块之外保持发散状态。可以使用显式的 `__syncwarp()` 来确保 warp 对后续指令已重新收敛。

13.2 分支谓词化

> 注意
> 低优先级：让编译器易于使用分支谓词化代替循环或控制语句。

有时，编译器可能会展开循环，或使用分支谓词化来优化掉 `if` 或 `switch` 语句。在这些情况下，没有任何 warp 会发散。程序员也可以使用以下方式控制循环展开：

代码块

```
1   #pragma unroll
```

有关此 pragma 的更多信息，请参考 CUDA C++ 编程指南。

当使用分支谓词化时，任何执行依赖于控制条件的指令都不会被跳过。相反，每条此类指令都与一个按线程的条件码或谓词相关联，该条件码或谓词根据控制条件被设置为 true 或 false。尽管每条指令都被调度执行，但只有谓词为 true 的指令才会实际执行。谓词为 false 的指令不会写入结果，也不会计算地址或读取操作数。

编译器仅在分支条件控制的指令数量小于或等于某个阈值时，才将分支指令替换为谓词化指令。

14. 部署 CUDA 应用程序

完成应用程序中一个或多个组件的 GPU 加速后，可以将结果与原始预期进行比较。回顾最初评估步骤，该步骤允许开发者确定加速给定热点可达到的加速比理论上限。

在处理其他热点以提高总体加速比之前，开发者应考虑将部分并行化的实现推进到生产环境中。这很重要，原因如下：例如，它允许用户尽早从投资中获益（加速可能只是部分的，但仍然有价值），并且通过为应用程序提供渐进式而非革命性的变更，最小化了开发者和用户的风险。

15. 理解编程环境

随着 NVIDIA 处理器的每一代更新，GPU 都会增加 CUDA 可以利用的新功能。因此，了解架构的特性非常重要。

程序员应注意两个版本号。第一个是计算能力，第二个是 CUDA Runtime 和 CUDA Driver API 的版本号。

15.1 CUDA 计算能力

计算能力描述了硬件的功能，反映了设备支持的指令集以及其他规格，例如每个 block 的最大线程数和每个多处理器的寄存器数。更高版本的计算能力是较低（即较早）版本的超集，因此它们是向后兼容的。

设备中 GPU 的计算能力可以通过编程方式查询，如 deviceQuery CUDA 示例所示。该程序的输出如图 16 所示。此信息通过调用 `cudaGetDeviceProperties()` 并访问其返回结构体中的信息来获取。

图 16 deviceQuery 报告的 CUDA 配置数据示例

计算能力的主修订号和次修订号显示在图 16 的第七行。该系统的设备 0 具有计算能力 7.0。

有关各种 GPU 计算能力的更多详细信息，请参见 CUDA C++ 编程指南中的 CUDA-Enabled GPUs 和 Compute Capabilities 部分。特别是，开发者应注意设备上的多处理器数量、寄存器数量和可用内存量，以及设备的任何特殊能力。

15.2 附加硬件数据

某些硬件功能不由计算能力描述。例如，内核执行与主机和设备之间的异步数据传输重叠的能力在大多数（但非所有）GPU 上可用，无论计算能力如何。在这种情况下，调用 `cudaGetDeviceProperties()` 来确定设备是否具有某种功能。例如，设备属性结构的 `asyncEngineCount` 字段指示内核执行和数据传输是否可能重叠（如果可能，还指示可以同时进行多少次传输）；同样，`canMapHostMemory` 字段指示是否可以执行零拷贝数据传输。

15.3 选择何计算能力目标

要针对特定版本的 NVIDIA 硬件和 CUDA 软件，请使用 nvcc 的 `-arch`、`-code` 和 `-gencode` 选项。例如，使用 warp shuffle 操作的代码必须使用 `-arch=sm_30`（或更高的计算能力）编译。

关于同时为多代 CUDA 设备构建代码的标志，请参见"构建最大兼容性"部分中的进一步讨论。

15.4 CUDA Runtime

CUDA 软件环境的主机运行时组件只能由主机函数使用。它提供处理以下功能的函数：

- 设备管理
- 上下文管理
- 内存管理

- 代码模块管理
- 执行控制
- 纹理引用管理
- 与 OpenGL 和 Direct3D 的互操作性

与较低级别的 CUDA Driver API 相比，CUDA Runtime 通过提供隐式初始化、上下文管理和设备代码模块管理，大大简化了设备管理。nvcc 生成的 C++ 主机代码使用 CUDA Runtime，因此链接到此代码的应用程序将依赖于 CUDA Runtime；类似地，任何使用 cuBLAS、cuFFT 和其他 CUDA Toolkit 库的代码也将依赖于 CUDA Runtime，这些库在内部使用 CUDA Runtime。

构成 CUDA Runtime API 的函数在 CUDA Toolkit 参考手册中有说明。

CUDA Runtime 处理内核加载以及在内核启动前设置内核参数和启动配置。隐式的驱动程序版本检查、代码初始化、CUDA 上下文管理、CUDA 模块管理（cubin 到函数映射）、内核配置和参数传递都由 CUDA Runtime 执行。

它包含两个主要部分：

- C 风格函数接口（`cuda_runtime_api.h`）
- 构建在 C 风格函数之上的 C++ 风格便利包装器（`cuda_runtime.h`）

有关 Runtime API 的更多信息，请参考 CUDA C++ 编程指南的 CUDA Runtime 部分。

16. CUDA 兼容性开发者指南

CUDA Toolkit 按月发布节奏交付新功能、性能改进和关键错误修复。CUDA 兼容性允许用户更新最新的 CUDA Toolkit 软件（包括编译器、库和工具），而无需更新整个驱动栈。

CUDA 软件环境由三部分组成：

- CUDA Toolkit（库、CUDA 运行时和开发者工具）—— 供开发者构建 CUDA 应用程序的 SDK
- CUDA 驱动 —— 用于运行 CUDA 应用程序的用户模式驱动组件（例如 Linux 系统上的 libcuda.so）
- NVIDIA GPU 设备驱动 —— NVIDIA GPU 的内核模式驱动组件

在 Linux 系统上，CUDA 驱动和内核模式组件捆绑在 NVIDIA 显示驱动包中。如图 1 所示。

图 17 CUDA 组件

CUDA 编译器（nvcc）提供了一种处理 CUDA 和非 CUDA 代码的方式（通过分离和引导编译），与 CUDA 运行时一起构成 CUDA 编译器工具链的一部分。CUDA Runtime API 为开发者提供高级 C++ 接口，用于简化设备管理、内核执行等，而 CUDA Driver API 则为应用程序提供低级编程接口以针对 NVIDIA 硬件。

构建在这些技术之上的是 CUDA 库，其中一些包含在 CUDA Toolkit 中，而另一些（如 cuDNN）可能独立于 CUDA Toolkit 发布。

16.1 CUDA Toolkit 版本控制

从 CUDA 11 开始，工具包版本基于业界标准的语义版本控制方案：X.Y.Z，其中：

- X 代表主版本 —— API 已更改，二进制兼容性被破坏
- Y 代表次版本 —— 引入新 API，弃用旧 API，源代码兼容性可能被破坏，但二进制兼容性得以维持
- Z 代表发布/补丁版本 —— 新的更新和补丁将递增此版本号

建议工具包中的每个组件都进行语义版本控制。从 CUDA 11.3 开始，NVRTC 也进行了语义版本控制。我们将在本文档后面说明其中一些。工具包中各组件的版本信息可在此表中找到。

CUDA 平台的兼容性旨在解决以下几种场景：

- 对正在生产中运行 GPU 的企业或数据中心系统升级 NVIDIA 驱动程序可能很复杂，可能需要提前规划。延迟推出新 NVIDIA 驱动程序可能意味着此类系统的用户可能无法访问 CUDA 版本中提供的新功能。新的 CUDA 版本不要求驱动程序更新，意味着新版本的软件可以更快地提供给用户。
- 许多构建在 CUDA 之上的软件库和应用程序（例如数学库或深度学习框架）与 CUDA 运行时、编译器或驱动程序没有直接依赖关系。在这些情况下，用户或开发者仍然可以从不必升级整个 CUDA Toolkit 或驱动程序以使用这些库或框架中受益。
- 升级依赖关系容易出错且耗时，在某些极端情况下，甚至可能改变程序的语义。不断使用最新的 CUDA Toolkit 重新编译意味着强制应用程序产品的最终客户进行升级。软件包管理器可以促进这一过程，但仍可能出现意外问题，如果发现错误，则需要重复上述升级过程。

CUDA 支持多种兼容性选择：

- 首次在 CUDA 10 中引入的 CUDA 前向兼容升级，旨在允许用户获取新的 CUDA 功能，并在具有较旧 NVIDIA 数据中心驱动安装的系统上运行使用新 CUDA 版本构建的应用程序。
- 首次在 CUDA 11.1 中引入的 CUDA 增强兼容性提供两大优势：
  - 通过利用 CUDA Toolkit 中各组件的语义版本控制，可以为某个 CUDA 次版本（例如 11.1）构建应用程序，并在主版本族内（即 11.x）的所有未来次版本中工作。
  - CUDA 运行时放宽了最低驱动程序版本检查，因此在迁移到新的次版本时不再需要驱动程序升级。

CUDA 驱动程序确保为已编译的 CUDA 应用程序保持向后二进制兼容性。使用早至 3.2 版本的 CUDA 工具包编译的应用程序将在较新的驱动程序上运行。

16.2 源代码兼容性

我们将源代码兼容性定义为库提供的一组保证，即针对特定版本库（使用 SDK）构建的良好形式的应用程序，在安装更新版本的 SDK 时将继续构建和运行而不会出错。

CUDA 驱动和 CUDA 运行时在不同 SDK 版本之间不具有源代码兼容性。API 可能被弃用和移除。因此，在旧版本工具包上成功编译的应用程序可能需要进行更改才能针对较新版本的工具包进行编译。

开发者通过弃用和文档机制收到有关当前或即将发生的更改的通知。这并不意味着使用旧版工具包编译的应用程序二进制文件将不再受支持。应用程序二进制文件依赖于 CUDA Driver API 接口，即使 CUDA Driver API 本身也可能在工具包版本之间发生变化，CUDA 保证 CUDA Driver API 接口的二进制兼容性。

16.3 二进制兼容性

我们将二进制兼容性定义为库提供的一组保证，即针对所述库的应用程序在动态链接到不同版本的库时将继续工作。

CUDA Driver API 具有带版本号的 C 风格 ABI，这保证在较旧驱动程序（例如 CUDA 3.2）上运行的应用程序将仍然在现代驱动程序（例如随 CUDA 11.0 提供的驱动程序）上运行并正常发挥功能。这意味着即使应用程序源代码可能需要进行更改才能针对较新的 CUDA Toolkit 重新编译以使用新功能，用较新版本替换系统中安装的驱动组件将始终支持现有应用程序及其功能。

因此，CUDA Driver API 是二进制兼容的（操作系统加载器可以拾取较新版本，应用程序继续工作），但不是源代码兼容的（针对较新 SDK 重新构建应用程序可能需要源代码更改）。

图 18 CUDA Toolkit 和最低驱动程序版本

在对本主题进行进一步讨论之前，开发者理解最低驱动程序版本的概念及其可能对自身产生的影响非常重要。

每个版本的 CUDA Toolkit（及运行时）都要求最低版本的 NVIDIA 驱动程序。针对某个 CUDA Toolkit 版本编译的应用程序只能在具有该工具包版本指定的最低驱动版本的系统中运行。在 CUDA 11.0 之前，工具包的最低驱动程序版本与该版本 CUDA Toolkit 随附的驱动程序版本相同。

因此，当使用 CUDA 11.0 构建应用程序时，它只能在具有 R450 或更高版本驱动程序的系统上运行。如果此类应用程序在安装了 R418 驱动程序的系统上运行，CUDA 初始化将返回错误，如下例所示。

在此示例中，deviceQuery 示例使用 CUDA 11.1 编译，并在具有 R418 的系统上运行。在此场景中，由于最低驱动程序要求，CUDA 初始化返回错误。

代码块

```
ubuntu@:~/samples/1_Utilities/deviceQuery
$ make
/usr/local/cuda-11.1/bin/nvcc -ccbin g++ -I../../common/inc -m64 -gencode
arch=compute_35,code=sm_35 -gencode arch=compute_37,code=sm_37 -gencode
arch=compute_50,code=sm_50 -gencode arch=compute_52,code=sm_52 -gencode
arch=compute_60,code=sm_60 -gencode arch=compute_61,code=sm_61 -gencode
arch=compute_70,code=sm_70 -gencode arch=compute_75,code=sm_75 -gencode
arch=compute_80,code=sm_80 -gencode arch=compute_86,code=sm_86 -gencode
arch=compute_86,code=compute_86 -o deviceQuery.o -c deviceQuery.cpp

/usr/local/cuda-11.1/bin/nvcc -ccbin g++ -m64 -gencode
arch=compute_35,code=sm_35 -gencode arch=compute_37,code=sm_37 -gencode
arch=compute_50,code=sm_50 -gencode arch=compute_52,code=sm_52 -gencode
arch=compute_60,code=sm_60 -gencode arch=compute_61,code=sm_61 -gencode
```

```
    arch=compute_70,code=sm_70 -gencode arch=compute_75,code=sm_75 -gencode
    arch=compute_80,code=sm_80 -gencode arch=compute_86,code=sm_86 -gencode
    arch=compute_86,code=compute_86 -o deviceQuery deviceQuery.o

$ nvidia-smi

+-----------------------------------------------------------------------------+
| NVIDIA-SMI 418.165.02   Driver Version: 418.165.02   CUDA Version: 10.1     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|===============================+======================+======================|
|   0  Tesla T4            On   | 00000000:00:1E.0 Off |                    0 |
| N/A   42C    P0    28W /  70W |      0MiB / 15079MiB |      0%      Default |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                       GPU Memory |
|  GPU       PID   Type   Process name                             Usage      |
|=============================================================================|
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+


$ samples/bin/x86_64/linux/release/deviceQuery
samples/bin/x86_64/linux/release/deviceQuery Starting...

 CUDA Device Query (Runtime API) version (CUDART static linking)

cudaGetDeviceCount returned 3
-> initialization error
Result = FAIL
```

有关最低驱动程序版本和工具包随附驱动程序版本的详细信息，请参考 CUDA Toolkit 发布说明。

16.3.1 CUDA 二进制（cubin）兼容性

一个稍微相关但重要的主题是 CUDA 中跨 GPU 架构的应用程序二进制兼容性。

CUDA C++ 为熟悉 C++ 编程语言的用户提供了一条简单途径，使其能够轻松编写供设备执行的程序。内核可以使用 CUDA 指令集架构（称为 PTX）编写，PTX 在 PTX 参考手册中有描述。然而，通常使用高级编程语言（如 C++）更为有效。在这两种情况下，内核都必须由 nvcc 编译为二进制代码（称为 cubin）才能在设备上执行。

cubin 是架构特定的。cubin 的二进制兼容性保证从一个计算能力次修订版到下一个，但不保证从一个计算能力次修订版到前一个，也不保证跨主计算能力修订版。换句话说，为计算能力 X.y 生成的 cubin 对象仅在计算能力为 X.z（其中 z≥y）的设备上执行。

要在特定计算能力的设备上执行代码，应用程序必须加载与该计算能力兼容的二进制或 PTX 代码。为了可移植性（即为了能够在具有更高计算能力的未来 GPU 架构上执行代码，而这些架构目前没有可以生成的二进制代码），应用程序必须加载 PTX 代码，该代码将由 NVIDIA 驱动程序为这些未来设备进行即时编译。

有关 cubin、PTX 和应用程序兼容性的更多信息，请参见 CUDA C++ 编程指南。

16.4 跨次版本的 CUDA 兼容性

通过利用语义版本控制，从 CUDA 11 开始，CUDA Toolkit 中的组件将在工具包的次版本之间保持二进制兼容。为了在次版本之间保持二进制兼容性，CUDA 运行时不再为每个次版本提高要求的最低驱动程序版本 —— 这仅在发布主版本时发生。

新工具链需要新的最低驱动程序的主要原因之一是处理 PTX 代码的 JIT 编译和二进制代码的 JIT 链接。

在本节中，我们将回顾在利用 CUDA 平台兼容性功能时可能需要新用户工作流的使用模式。

16.4.1 在 CUDA 次版本内的现有 CUDA 应用程序

代码块

```
$ nvidia-smi

+-----------------------------------------------------------------------------+
| NVIDIA-SMI 450.80.02    Driver Version: 450.80.02    CUDA Version: 11.0     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  Tesla T4            On   | 00000000:00:1E.0 Off |                    0 |
| N/A   39C    P8     9W /  70W |      0MiB / 15109MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                                  |
|  GPU   GI   CI        PID   Type   Process name                  GPU Memory |
|        ID   ID                                                   Usage      |
|=============================================================================|
|  No running processes found                                                 |
+-----------------------------------------------------------------------------+
```

当我们的 CUDA 11.1 应用程序（即静态链接 cudart 11.1）在系统上运行时，我们看到即使驱动程序报告 11.0 版本，它也能成功运行 —— 也就是说，不需要更新系统上的驱动程序或其他工具包组件。

```
代码块
$ samples/bin/x86_64/linux/release/deviceQuery
samples/bin/x86_64/linux/release/deviceQuery Starting...

 CUDA Device Query (Runtime API) version (CUDART static linking)

Detected 1 CUDA Capable device(s)

Device 0: "Tesla T4"
  CUDA Driver Version / Runtime Version          11.0 / 11.1
  CUDA Capability Major/Minor version number:    7.5

  ...<snip>...

deviceQuery, CUDA Driver = CUDART, CUDA Driver Version = 11.0, CUDA Runtime
Version = 11.1, NumDevs = 1
Result = PASS
```

通过使用新的 CUDA 版本，用户可以从新的 CUDA 编程模型 API、编译器优化和数学库功能中受益。

以下各节讨论一些注意事项和考量因素。

16.4.1.1 处理新的 CUDA 功能和 Driver API

一部分 CUDA API 不需要新的驱动程序，它们都可以在没有任何驱动程序依赖的情况下使用。例如，`cuMemMap` API 或 CUDA 11.0 之前引入的任何 API（如 `cudaDeviceSynchronize`）不需要驱动程序升级。要使用在次版本中引入的其他 CUDA API（需要新的驱动程序），必须实现回退或优雅失败。这种情况与目前开发者使用宏根据 CUDA 版本编译掉功能的情况没有区别。用户应参考 CUDA 头文件和文档以了解某个版本中引入的新 CUDA API。

当使用工具包次版本中暴露的功能时，如果应用程序在较旧的 CUDA 驱动程序上运行，该功能可能在运行时不可用。希望利用此类功能的用户应在代码中使用动态检查来查询其可用性：

```
代码块
static bool hostRegisterFeatureSupported = false;
static bool hostRegisterIsDeviceAddress = false;

static error_t cuFooFunction(int *ptr)
{
    int *dptr = null;
    if (hostRegisterFeatureSupported) {
         cudaHostRegister(ptr, size, flags);
         if (hostRegisterIsDeviceAddress) {
              qptr = ptr;
         }
       else {
          cudaHostGetDevicePointer(&qptr, ptr, 0);
```

```
            }
        }
    else {
            // cudaMalloc();
            // cudaMemcpy();
        }
    gemm<<<1,1>>>(dptr);
    cudaDeviceSynchronize();
}

int main()
{
    // rest of code here
    cudaDeviceGetAttribute(
           &hostRegisterFeatureSupported,
           cudaDevAttrHostRegisterSupported,
           0);
    cudaDeviceGetAttribute(
           &hostRegisterIsDeviceAddress,
           cudaDevAttrCanUseHostPointerForRegisteredMem,
           0);
    cuFooFunction(/* malloced pointer */);
}
```

或者，如果没有新的 CUDA 驱动程序，应用程序的接口可能根本无法工作，那么最好立即返回错误：

代码块

```
#define MIN_VERSION 11010
cudaError_t foo()
{
    int version = 0;
    cudaGetDriverVersion(&version);
    if (version < MIN_VERSION) {
        return CUDA_ERROR_INSUFFICIENT_DRIVER;
    }
    // proceed as normal
}
```

新增了一个错误码来指示您正在运行的驱动程序缺少该功能：`cudaErrorCallRequiresNewerDriver`。

16.4.1.2 使用 PTX

PTX 定义了一个用于通用并行线程执行的虚拟机和 ISA。PTX 程序在加载时通过 JIT 编译器（属于 CUDA 驱动程序的一部分）转换为目标硬件指令集。由于 PTX 是由 CUDA 驱动程序编译的，新工具链

将生成与旧版 CUDA 驱动程序不兼容的 PTX。当 PTX 用于未来设备兼容性（最常见的情况）时，这不是问题，但当用于运行时编译时可能导致问题。

对于继续使用 PTX 的代码，为了支持在较旧的驱动程序上进行编译，代码必须首先通过静态的 ptxjitcompiler 库或 NVRTC 转换为设备代码，并选择为特定架构（例如 sm_80）而不是虚拟架构（例如 compute_80）生成代码。对于此工作流，CUDA Toolkit 随附了一个新的 `nvptxcompiler_static` 库。

我们可以在以下示例中看到此用法：

代码块

```
char* compilePTXToNVElf()
{
    nvPTXCompilerHandle compiler = NULL;
    nvPTXCompileResult status;

    size_t elfSize, infoSize, errorSize;
    char *elf, *infoLog, *errorLog;
    int minorVer, majorVer;

    const char* compile_options[] = { "--gpu-name=sm_80",
                                      "--device-debug"
    };

    nvPTXCompilerGetVersion(&majorVer, &minorVer);
    nvPTXCompilerCreate(&compiler, (size_t)strlen(ptxCode), ptxCode);
    status = nvPTXCompilerCompile(compiler, 2, compile_options);
    if (status != NVPTXCOMPILE_SUCCESS) {
        nvPTXCompilerGetErrorLogSize(compiler, (void*)&errorSize);

        if (errorSize != 0) {
            errorLog = (char*)malloc(errorSize+1);
            nvPTXCompilerGetErrorLog(compiler, (void*)errorLog);
            printf("Error log: %s\n", errorLog);
            free(errorLog);
        }
        exit(1);
    }

    nvPTXCompilerGetCompiledProgramSize(compiler, &elfSize));
    elf = (char*)malloc(elfSize);
    nvPTXCompilerGetCompiledProgram(compiler, (void*)elf);
    nvPTXCompilerGetInfoLogSize(compiler, (void*)&infoSize);

    if (infoSize != 0) {
        infoLog = (char*)malloc(infoSize+1);
        nvPTXCompilerGetInfoLog(compiler, (void*)infoLog);
```

```
        printf("Info log: %s\n", infoLog);
        free(infoLog);
    }

    nvPTXCompilerDestroy(&compiler);
    return elf;
}
```

16.4.1.3 动态代码生成

NVRTC 是 CUDA C++ 的运行时编译库。它接受字符串形式的 CUDA C++ 源代码，并创建可用于获取 PTX 的句柄。NVRTC 生成的 PTX 字符串可以通过 `cuModuleLoadData` 和 `cuModuleLoadDataEx` 加载。

尚不支持处理可重定位对象，因此 CUDA 驱动程序中的 `cuLink*` 系列 API 在增强兼容性下无法工作。这些 API 当前需要与 CUDA 运行时版本匹配的升级驱动程序。

如 PTX 部分所述，将 PTX 编译为设备代码是伴随 CUDA 驱动程序进行的，因此生成的 PTX 可能比部署系统上的驱动程序所支持的版本更新。使用 NVRTC 时，建议通过 PTX 用户工作流中概述的步骤先将生成的 PTX 代码转换为最终设备代码。这可以确保您的代码兼容。或者，从 CUDA 11.1 开始，NVRTC 可以直接生成 cubin。使用新 API 的应用程序可以直接使用驱动程序 API `cuModuleLoadData` 和 `cuModuleLoadDataEx` 加载最终设备代码。

NVRTC 过去仅通过 `-arch` 选项支持虚拟架构，因为它只生成 PTX。现在它也将支持实际架构以生成 SASS。接口已增强，如果指定了实际架构，可以检索 PTX 或 cubin。

以下示例展示了如何使用新功能调整现有示例，在此示例中由 `USE_CUBIN` 宏控制：

代码块

```
#include <nvrtc.h>
#include <cuda.h>
#include <iostream>

void NVRTC_SAFE_CALL(nvrtcResult result) {
  if (result != NVRTC_SUCCESS) {
    std::cerr << "\nnvrtc error: " << nvrtcGetErrorString(result) << '\n';
    std::exit(1);
  }
}

void CUDA_SAFE_CALL(CUresult result) {
  if (result != CUDA_SUCCESS) {
    const char *msg;
    cuGetErrorName(result, &msg);
    std::cerr << "\ncuda error: " << msg << '\n';
    std::exit(1);
```

```
  }
}

const char *hello = "                                                \n\
extern \"C\" __global__ void hello() {                               \n\
  printf(\"hello world\\n\");                                        \n\
}                                                                    \n";

int main()
{
  nvrtcProgram prog;
  NVRTC_SAFE_CALL(nvrtcCreateProgram(&prog, hello, "hello.cu", 0, NULL, NULL));
#ifdef USE_CUBIN
  const char *opts[] = {"-arch=sm_70"};
#else
  const char *opts[] = {"-arch=compute_70"};
#endif
  nvrtcResult compileResult = nvrtcCompileProgram(prog, 1, opts);
  size_t logSize;
  NVRTC_SAFE_CALL(nvrtcGetProgramLogSize(prog, &logSize));
  char *log = new char[logSize];
  NVRTC_SAFE_CALL(nvrtcGetProgramLog(prog, log));
  std::cout << log << '\n';
  delete[] log;
  if (compileResult != NVRTC_SUCCESS)
    exit(1);
  size_t codeSize;
#ifdef USE_CUBIN
  NVRTC_SAFE_CALL(nvrtcGetCUBINSize(prog, &codeSize));
  char *code = new char[codeSize];
  NVRTC_SAFE_CALL(nvrtcGetCUBIN(prog, code));
#else
  NVRTC_SAFE_CALL(nvrtcGetPTXSize(prog, &codeSize));
  char *code = new char[codeSize];
  NVRTC_SAFE_CALL(nvrtcGetPTX(prog, code));
#endif
  NVRTC_SAFE_CALL(nvrtcDestroyProgram(&prog));
  CUdevice cuDevice;
  CUcontext context;
  CUmodule module;
  CUfunction kernel;
  CUDA_SAFE_CALL(cuInit(0));
  CUDA_SAFE_CALL(cuDeviceGet(&cuDevice, 0));
  CUDA_SAFE_CALL(cuCtxCreate(&context, NULL, 0, cuDevice));
  CUDA_SAFE_CALL(cuModuleLoadDataEx(&module, code, 0, 0, 0));
  CUDA_SAFE_CALL(cuModuleGetFunction(&kernel, module, "hello"));
  CUDA_SAFE_CALL(cuLaunchKernel(kernel, 1, 1, 1, 1, 1, 1, 0, NULL, NULL, 0));
```

```
65      CUDA_SAFE_CALL(cuCtxSynchronize());
66      CUDA_SAFE_CALL(cuModuleUnload(module));
67      CUDA_SAFE_CALL(cuCtxDestroy(context));
68      delete[] code;
69    }
```

16.4.1.4 构建次版本兼容库的建议

建议静态链接 CUDA 运行时以最小化依赖。验证您的库不会在已建立的 ABI 契约之外泄漏依赖、破坏、命名空间等。

为库的 soname 遵循语义版本控制。拥有语义版本化的 ABI 意味着接口需要维护和版本化。库应遵循语义规则，在对 ABI 契约产生影响时递增版本号。缺少依赖关系也是一种二进制兼容性破坏，因此您应为依赖于这些接口的功能提供回退或保护措施。当有 ABI 破坏性更改（如 API 弃用和修改）时，递增主版本号。新 API 可以在次版本中添加。

有条件地使用功能以保持与旧驱动程序的兼容性。如果不使用新功能（或者有条件地使用并提供回退），您将能够保持兼容。

不要暴露可能更改的 ABI 结构。使用指向带有嵌入大小字段的结构体的指针是更好的解决方案。

当与工具包中的动态库链接时，该库必须等于或高于应用程序链接中涉及的任何组件所需的版本。例如，如果您链接到 CUDA 11.1 动态运行时并使用 11.1 的功能，同时链接到一个单独的、链接到 CUDA 11.2 动态运行时并需要 11.2 功能的共享库，则最终链接步骤必须包含 CUDA 11.2 或更新的动态运行时。

16.4.1.5 在应用程序中利用次版本兼容性的建议

某些功能可能不可用，因此应在适用时进行查询。这对于构建与 GPU 架构、平台和编译器无关的应用程序来说很常见。然而，我们现在将"底层驱动程序"也加入这一组合。

与上一节关于库构建建议的内容一样，如果使用 CUDA 运行时，我们建议在构建应用程序时静态链接到 CUDA 运行时。当直接使用 Driver API 时，我们建议使用此处介绍的新驱动程序入口点访问 API（`cuGetProcAddress`）：CUDA Driver API :: CUDA Toolkit Documentation。

当使用共享或静态库时，请遵循该库的发布说明以确定该库是否支持次版本兼容性。

17. 准备部署

17.1 测试 CUDA 可用性

在部署 CUDA 应用程序时，通常希望确保即使目标机器没有配备支持 CUDA 的 GPU 和/或安装了足够版本的 NVIDIA 驱动程序，应用程序也能继续正常运行。（针对具有已知配置的单台机器的开发者可以选择跳过本节。）

检测支持 CUDA 的 GPU

当应用程序将部署到具有任意/未知配置的目标机器时，应用程序应显式测试是否存在支持 CUDA 的 GPU，以便在没有此类设备时采取适当措施。`cudaGetDeviceCount()` 函数可用于查询可用设备数量。与所有 CUDA Runtime API 函数一样，如果没有支持 CUDA 的 GPU，此函数将优雅失败并向应用程序返回 `cudaErrorNoDevice`；如果没有安装适当版本的 NVIDIA 驱动程序，则返回 `cudaErrorInsufficientDriver`。如果 `cudaGetDeviceCount()` 报告错误，应用程序应回退到替代代码路径。

具有多个 GPU 的系统可能包含不同硬件版本和能力的 GPU。当同一应用程序使用多个 GPU 时，建议使用相同类型的 GPU，而不是混合硬件代际。`cudaChooseDevice()` 函数可用于选择最匹配所需功能集的设备。

检测硬件和软件配置

当应用程序依赖于某些硬件或软件功能的可用性来启用特定功能时，可以查询 CUDA API 以获取有关可用设备配置和已安装软件版本的详细信息。

`cudaGetDeviceProperties()` 函数报告可用设备的各项功能，包括设备的 CUDA 计算能力（另请参见 CUDA C++ 编程指南的 Compute Capabilities 部分）。有关如何查询可用 CUDA 软件 API 版本的详细信息，请参见 Version Management。

17.2 错误处理

所有 CUDA Runtime API 调用都返回一个类型为 `cudaError_t` 的错误码；如果没有发生错误，返回值将等于 `cudaSuccess`。（例外情况是内核启动（返回 void）和 `cudaGetErrorString()`（返回描述传入的 `cudaError_t` 码的字符串）。）CUDA Toolkit 库（cuBLAS、cuFFT 等）同样返回其自己的错误码集。

由于某些 CUDA API 调用和所有内核启动相对于主机代码是异步的，错误也可能异步报告给主机；通常这发生在主机和设备彼此同步的下一个时机，例如在调用 `cudaMemcpy()` 或 `cudaDeviceSynchronize()` 期间。

始终检查所有 CUDA API 函数的错误返回值，即使是对那些预期不会失败的函数也是如此，因为这将允许应用程序在错误发生时尽快检测并恢复。要检查使用 `<<<...>>>` 语法启动的内核中发生的错误（该语法不返回任何错误码），应紧跟在内核启动之后检查 `cudaGetLastError()` 的返回码。不检查 CUDA API 错误的应用程序有时可能会运行到完成，却没有注意到 GPU 计算的数据是不完整、无效或未初始化的。

> 注意
> CUDA Toolkit 示例提供了几个辅助函数，用于对各种 CUDA API 进行错误检查；这些辅助函数位于 CUDA Toolkit 的 samples/common/inc/helper_cuda.h 文件中。

17.3 构建最大兼容性

每一代支持 CUDA 的设备都有一个关联的计算能力版本，指示该设备支持的功能集（参见 CUDA 计算能力）。在构建文件时，可以向 nvcc 编译器指定一个或多个计算能力版本；针对应用程序目标 GPU

的原生计算能力进行编译，对于确保应用程序内核获得最佳性能并能够使用给定代 GPU 上可用的功能至关重要。

当应用程序同时为多种计算能力构建时（对 nvcc 使用多个 `-gencode` 标志实例），指定计算能力的二进制文件被合并到可执行文件中，CUDA Driver 在运行时根据当前设备的计算能力选择最合适的二进制文件。如果适当的原生二进制文件（cubin）不可用，但中间 PTX 代码（针对抽象虚拟指令集，用于前向兼容性）可用，则内核将通过即时编译（JIT）（参见编译器 JIT 缓存管理工具）从 PTX 编译为该设备的原生 cubin。如果 PTX 也不可用，则内核启动将失败。

Windows

代码块

```
nvcc.exe -ccbin "C:\vs2008\VC\bin"
  -Xcompiler "/EHsc /W3 /nologo /O2 /Zi /MT"
  -gencode=arch=compute_30,code=sm_30
  -gencode=arch=compute_35,code=sm_35
  -gencode=arch=compute_50,code=sm_50
  -gencode=arch=compute_60,code=sm_60
  -gencode=arch=compute_70,code=sm_70
  -gencode=arch=compute_75,code=sm_75
  -gencode=arch=compute_75,code=compute_75
  --compile -o "Release\mykernel.cu.obj" "mykernel.cu"
```

Mac/Linux

代码块

```
/usr/local/cuda/bin/nvcc
  -gencode=arch=compute_30,code=sm_30
  -gencode=arch=compute_35,code=sm_35
  -gencode=arch=compute_50,code=sm_50
  -gencode=arch=compute_60,code=sm_60
  -gencode=arch=compute_70,code=sm_70
  -gencode=arch=compute_75,code=sm_75
  -gencode=arch=compute_75,code=compute_75
  -O2 -o mykernel.o -c mykernel.cu
```

或者，nvcc 命令行选项 `-arch=sm_XX` 可以用作以下更显式的 `-gencode=` 命令行选项的简写等价形式：

代码块

```
-gencode=arch=compute_XX,code=sm_XX
-gencode=arch=compute_XX,code=compute_XX
```

然而，虽然 `-arch=sm_XX` 命令行选项确实默认包含了 PTX 后端目标（由于它隐含的 `code=compute_XX` 目标），但它每次只能指定一个目标 cubin 架构，并且不能在同一个 nvcc 命令行上使用多个 `-arch=` 选项，这就是为什么上面的示例显式使用 `-gencode=` 的原因。

17.4 分发 CUDA Runtime 和库

CUDA 应用程序是针对 CUDA Runtime 库构建的，该库处理设备、内存和内核管理。与 CUDA 驱动程序不同，CUDA Runtime 不保证跨版本的向前或向后二进制兼容性。因此，最好在使用动态链接时将 CUDA Runtime 库与应用程序一起重新分发，或者静态链接到 CUDA Runtime。这将确保即使用户没有安装与构建应用程序时相同的 CUDA Toolkit，可执行文件也能运行。

> 注意
> 当静态链接到 CUDA Runtime 时，多个版本的运行时可以在同一应用程序进程中和平共存；例如，如果应用程序使用一个版本的 CUDA Runtime，而该应用程序的插件静态链接到不同版本，这是完全可以接受的，只要安装的 NVIDIA 驱动程序对两者都足够即可。

静态链接的 CUDA Runtime

最简单的选择是静态链接到 CUDA Runtime。这是使用 nvcc 链接时在 CUDA 5.5 及更高版本中的默认方式。静态链接使可执行文件稍大，但确保正确版本的运行时库函数包含在应用程序二进制文件中，而无需单独重新分发 CUDA Runtime 库。

动态链接的 CUDA Runtime

如果由于某种原因静态链接到 CUDA Runtime 不切实际，则也可以使用动态链接版本的 CUDA Runtime 库。（这是 CUDA 5.0 及更早版本中提供的默认且唯一的选项。）

要在使用 CUDA 5.5 或更高版本中的 nvcc 链接应用程序时使用 CUDA Runtime 的动态链接，请在链接命令行中添加 `--cudart=shared` 标志；否则默认使用静态链接的 CUDA Runtime 库。

在应用程序动态链接到 CUDA Runtime 之后，此版本的运行时库应与应用程序捆绑在一起。它可以复制到与应用程序可执行文件相同的目录中，或复制到该安装路径的子目录中。

其他 CUDA 库

虽然 CUDA Runtime 提供静态链接选项，但 CUDA Toolkit 中包含的某些库仅以动态链接形式提供。与动态链接版本的 CUDA Runtime 库一样，在分发应用程序时，这些库应与应用程序可执行文件捆绑在一起。

17.4.1 CUDA Toolkit 库再分发

CUDA Toolkit 的最终用户许可协议（EULA）允许在特定条款和条件下再分发许多 CUDA 库。这使得依赖这些库的应用程序可以重新分发其构建和测试时所使用的确切库版本，从而避免对可能安装了不同版本 CUDA Toolkit（或根本没有安装）的最终用户造成任何麻烦。详情请参阅 EULA。

> 注意
> 这不适用于 NVIDIA 驱动程序；最终用户仍必须下载并安装适合其 GPU 和操作系统的 NVIDIA 驱动程序。

17.4.1.1 应再分发哪些文件

当重新分发一个或多个 CUDA 库的动态链接版本时，重要的是确定需要再分发的确切文件。以下示例使用 CUDA Toolkit 5.5 中的 cuBLAS 库作为说明：

Linux

在 Linux 上的共享库中，有一个称为 SONAME 的字符串字段，指示库的二进制兼容性级别。应用程序构建时所针对的库的 SONAME 必须与随应用程序再分发的库的文件名匹配。

例如，在标准 CUDA Toolkit 安装中，文件 `libcublas.so` 和 `libcublas.so.5.5` 都是指向 cuBLAS 特定构建版本的符号链接，其命名格式为 `libcublas.so.5.5.x`，其中 x 是构建编号（例如 `libcublas.so.5.5.17`）。然而，此库的 SONAME 为"libcublas.so.5.5"：

代码块

```
$ objdump -p /usr/local/cuda/lib64/libcublas.so | grep SONAME
    SONAME               libcublas.so.5.5
```

因此，即使在链接应用程序时使用了 `-lcublas`（未指定版本号），链接时找到的 SONAME 意味着"libcublas.so.5.5"是动态加载器在加载应用程序时将查找的文件名，因此必须是随应用程序再分发的文件的名称（或指向相同文件的符号链接）。

`ldd` 工具对于识别应用程序期望在运行时查找的库的确切文件名，以及在当前库搜索路径下动态加载器在加载应用程序时会选择的该库副本的路径（如果有的话）非常有用：

代码块

```
$ ldd a.out | grep libcublas
    libcublas.so.5.5 => /usr/local/cuda/lib64/libcublas.so.5.5
```

Mac

在 Mac OS X 上的共享库中，有一个称为安装名称的字段，指示库预期的安装路径和文件名；CUDA 库也使用此文件名来指示二进制兼容性。此字段的值会传播到针对该库构建的应用程序中，并用于在运行时定位正确版本的库。

例如，如果 cuBLAS 库的安装名称指定为 `@rpath/libcublas.5.5.dylib`，则该库的版本为 5.5，随应用程序再分发的该库的副本必须命名为 `libcublas.5.5.dylib`，即使在链接时仅使用了 `-lcublas`（未指定版本号）。

此外，此文件应安装到应用程序的 `@rpath` 中；参见"Redistributed CUDA Libraries 的安装位置"。

要查看库的安装名称，请使用 `otool -L` 命令：

代码块

```
$ otool -L a.out
```

```
a.out:
        @rpath/libcublas.5.5.dylib (...)
```

Windows

Windows 上 CUDA 库的二进制兼容性版本通过文件名的一部分来指示。

例如，链接到 cuBLAS 5.5 的 64 位应用程序将在运行时查找 `cublas64_55.dll`，因此这是应该随该应用程序再分发的文件，尽管 `cublas.lib` 是应用程序链接时所针对的文件。对于 32 位应用程序，文件名将是 `cublas32_55.dll`。

要验证应用程序期望在运行时查找的确切 DLL 文件名，请使用 Visual Studio 命令提示符中的 `dumpbin` 工具：

代码块

```
$ dumpbin /IMPORTS a.exe
Microsoft (R) COFF/PE Dumper Version 10.00.40219.01
Copyright (C) Microsoft Corporation.  All rights reserved.


Dump of file a.exe

File Type: EXECUTABLE IMAGE

  Section contains the following imports:

    ...
    cublas64_55.dll
    ...
```

17.4.1.2 Redistributed CUDA Libraries 的安装位置

一旦确定了用于再分发的正确库文件，就必须配置它们安装到一个应用程序能够找到它们的位置。

在 Windows 上，如果 CUDA Runtime 或其他动态链接的 CUDA Toolkit 库放置在可执行文件所在的同一目录中，Windows 将自动定位它。在 Linux 和 Mac 上，应使用 `-rpath` 链接器选项来指示可执行文件在搜索系统路径之前先在其本地路径中搜索这些库：

Linux/Mac

代码块

```
nvcc -I $(CUDA_HOME)/include
  -Xlinker "-rpath '$ORIGIN'" --cudart=shared
  -o myprogram myprogram.cu
```

Windows

```
代码块
1   nvcc.exe -ccbin "C:\vs2008\VC\bin"
2     -Xcompiler "/EHsc /W3 /nologo /O2 /Zi /MT" --cudart=shared
3     -o "Release\myprogram.exe" "myprogram.cu"
```

> 注意
> 可能需要调整 -ccbin 的值以反映您的 Visual Studio 安装位置。

要指定将分发库的替代路径，请使用类似下面的链接器选项：

Linux/Mac

```
代码块
1   nvcc -I $(CUDA_HOME)/include
2     -Xlinker "-rpath '$ORIGIN/lib'" --cudart=shared
3     -o myprogram myprogram.cu
```

Windows

```
代码块
1   nvcc.exe -ccbin "C:\vs2008\VC\bin"
2     -Xcompiler "/EHsc /W3 /nologo /O2 /Zi /MT /DELAY" --cudart=shared
3     -o "Release\myprogram.exe" "myprogram.cu"
```

对于 Linux 和 Mac，如前所述使用 `-rpath` 选项。对于 Windows，使用 `/DELAY` 选项；这要求应用程序在首次调用任何 CUDA API 函数之前调用 `SetDllDirectory()` ，以指定包含 CUDA DLL 的目录。

> 注意
> 对于 Windows 8，应使用 SetDefaultDLLDirectories() 和 AddDllDirectory() 代替 SetDllDirectory()。有关这些例程的更多信息，请参见 MSDN 文档。

18. 部署基础设施工具

18.1 Nvidia-SMI

NVIDIA 系统管理接口（nvidia-smi）是一个命令行实用程序，用于辅助管理和监控 NVIDIA GPU 设备。此实用程序允许管理员查询 GPU 设备状态，并且通过适当的权限，允许管理员修改 GPU 设备状态。nvidia-smi 主要针对 Tesla 和某些 Quadro GPU，尽管对其他 NVIDIA GPU 也提供有限支持。nvidia-smi 随 Linux 上的 NVIDIA GPU 显示驱动程序以及 64 位 Windows Server 2008 R2 和

Windows 7 一起发布。nvidia-smi 可以将查询信息以 XML 或人类可读的纯文本格式输出到标准输出或文件中。详情请参见 nvidia-smi 文档。请注意，不保证新版本的 nvidia-smi 与先前版本向后兼容。

18.1.1 可查询的状态

ECC 错误计数：报告可纠正的单比特错误和可检测的双比特错误。错误计数针对当前启动周期和 GPU 的整个生命周期提供。

GPU 利用率：报告 GPU 计算资源和内存接口的当前利用率。

活跃的计算进程：报告在 GPU 上运行的活跃进程列表，以及相应的进程名称/ID 和分配的 GPU 内存。

时钟和性能状态：报告多个重要时钟域的最大和当前时钟频率，以及当前的 GPU 性能状态（pstate）。

温度和风扇速度：报告当前 GPU 核心温度，对于配备主动冷却的产品，还报告风扇速度。

电源管理：报告当前板卡功耗和功率限制，适用于报告这些测量值的产品。

识别信息：报告各种动态和静态信息，包括板卡序列号、PCI 设备 ID、VBIOS/Inforom 版本号和产品名称。

18.1.2 可修改的状态

ECC 模式：启用和禁用 ECC 报告。

ECC 重置：清除单比特和双比特 ECC 错误计数。

计算模式：指示计算进程是否可以在 GPU 上运行，以及它们是独占运行还是与其他计算进程并发运行。

持久模式：指示当没有应用程序连接到 GPU 时，NVIDIA 驱动程序是否保持加载状态。在大多数情况下，最好启用此选项。

GPU 重置：通过辅助总线重置重新初始化 GPU 硬件和软件状态。

18.2 NVML

NVIDIA 管理库（NVML）是一个基于 C 的接口，提供对通过 nvidia-smi 暴露的查询和命令的直接访问，旨在作为构建第三方系统管理应用程序的平台。NVML API 随 CUDA Toolkit（从版本 8.0 开始）一起提供，也可在 NVIDIA 开发者网站上作为 GPU 部署工具包的一部分独立获取，通过单个头文件提供，并附有 PDF 文档、存根库和示例应用程序；参见 https://developer.nvidia.com/gpu-deployment-kit。每个新版本的 NVML 是向后兼容的。

为 NVML API 提供了一套额外的 Perl 和 Python 绑定。这些绑定暴露与基于 C 的接口相同的功能，并且也提供向后兼容性。Perl 绑定通过 CPAN 提供，Python 绑定通过 PyPI 提供。

所有这些产品（nvidia-smi、NVML 和 NVML 语言绑定）随每个新的 CUDA 版本更新，并提供大致相同的功能。

有关更多信息，请参见 https://developer.nvidia.com/nvidia-management-library-nvml。

18.3 集群管理工具

管理您的 GPU 集群将有助于实现最大的 GPU 利用率，并帮助您和您的用户获得最佳性能。业界许多最流行的集群管理工具通过 NVML 支持 CUDA GPU。有关其中一些工具的列表，请参见 https://developer.nvidia.com/cluster-management。

18.4 编译器 JIT 缓存管理工具

应用程序在运行时加载的任何 PTX 设备代码都会由设备驱动程序进一步编译为二进制代码。这称为即时编译（JIT）。即时编译增加了应用程序加载时间，但允许应用程序受益于最新的编译器改进。这也是应用程序能够在编译时不存在的设备上运行的唯一方式。

当使用 PTX 设备代码的 JIT 编译时，NVIDIA 驱动程序将生成的二进制代码缓存到磁盘上。此行为的某些方面（如缓存位置和最大缓存大小）可以通过使用环境变量来控制；参见 CUDA C++ 编程指南的 Just in Time Compilation 部分。

18.5 CUDA_VISIBLE_DEVICES

可以通过 `CUDA_VISIBLE_DEVICES` 环境变量，在 CUDA 应用程序启动之前重新排列已安装的 CUDA 设备集合，这些设备将对 CUDA 应用程序可见并被枚举。

要使应用程序可见的设备应以系统范围可枚举设备列表中的逗号分隔列表形式包含。例如，要仅使用系统范围设备列表中的设备 0 和 2，请在启动应用程序之前设置 `CUDA_VISIBLE_DEVICES=0,2`。然后，应用程序将这些设备分别枚举为设备 0 和设备 1。

19. 建议和最佳实践

本章包含了本文档中说明的优化建议的摘要。

19.1 总体性能优化策略

性能优化围绕三个基本策略展开：

- 最大化并行执行
- 优化内存使用以实现最大内存带宽
- 优化指令使用以实现最大指令吞吐量

最大化并行执行始于以尽可能暴露更多并行性的方式构建算法。一旦算法的并行性被暴露，就需要尽可能高效地将其映射到硬件。这通过仔细选择每个内核启动的执行配置来实现。应用程序还应通过显式地通过流暴露设备上的并发执行，以及最大化主机和设备之间的并发执行，在更高级别上最大化并行执行。

优化内存使用始于最小化主机和设备之间的数据传输，因为这些传输的带宽远低于设备内部的数据传输带宽。内核对全局内存的访问也应通过最大化设备上共享内存的使用来最小化。有时，最佳优化甚至可能是通过在任何需要时简单地重新计算数据来避免任何数据传输。

有效带宽可以根据每种内存类型的访问模式变化一个数量级。因此，优化内存使用的下一步是根据最佳内存访问模式来组织内存访问。这种优化对全局内存访问尤其重要，因为访问延迟需要数百个时钟周期。相比之下，共享内存访问通常仅在存在高度 bank 冲突时才值得优化。

至于优化指令使用，应避免使用吞吐量低的算术指令。这意味着在不影响最终结果的情况下，用精度换取速度，例如使用内建函数代替常规函数，或使用单精度代替双精度。最后，由于设备的 SIMT（单指令多线程）特性，必须特别关注控制流指令。

20. nvcc 编译器开关

20.1 nvcc

NVIDIA nvcc 编译器驱动程序将 .cu 文件转换为主机系统的 C++ 和设备系统的 CUDA 汇编或二进制指令。它支持多个命令行参数，其中以下参数对优化和相关最佳实践特别有用：

- `-maxrregcount=N` 在每文件级别指定内核可以使用的最大寄存器数。参见寄存器压力。（另请参阅 CUDA C++ 编程指南中 Execution Configuration 部分讨论的 `__launch_bounds__` 限定符，以按内核控制寄存器使用数量。）
- `--ptxas-options=-v` 或 `-Xptxas=-v` 列出每个内核的寄存器、共享和常量内存使用情况
- `-ftz=true`（非规范化数刷新为零）
- `-prec-div=false`（精度较低的除法）
- `-prec-sqrt=false`（精度较低的平方根）
- `-use_fast_math` 编译器选项将每个 `functionName()` 调用强制替换为等效的 `__functionName()` 调用。这使得代码运行更快，代价是精度和准确性降低。参见数学库。

21. 声明

21.1 声明

本文档仅供参考，不得视为对产品特定功能、条件或质量的保证。NVIDIA Corporation（"NVIDIA"）不对本文档所含信息的准确性或完整性作任何明示或暗示的陈述或保证，并且对本文档中可能包含的任何错误不承担任何责任。NVIDIA 对使用此类信息导致的后果或可能导致的任何第三方专利或其他权利侵犯不承担任何责任。本文档并非开发、发布或交付任何材料（定义见下文）、代码或功能的承诺。

NVIDIA 保留随时对本文档进行更正、修改、增强、改进和任何其他更改的权利，恕不另行通知。

客户在下订单之前应获取最新的相关信息，并应验证该信息是当前且完整的。

NVIDIA 产品根据 NVIDIA 在订单确认时提供的标准销售条款和条件进行销售，除非经 NVIDIA 和客户的授权代表签署的单独销售协议另有约定（"销售条款"）。NVIDIA 特此明确反对客户就购买本文档中提及的 NVIDIA 产品适用任何客户通用条款和条件。本文档既不直接也不间接形成任何合同义务。

NVIDIA 产品未设计、授权或保证适用于医疗、军事、航空、航天或生命支持设备，也不适用于 NVIDIA 产品的故障或失效可能合理预期导致人身伤害、死亡或财产或环境损害的应用。NVIDIA 对在此类设备或应用中包含和/或使用 NVIDIA 产品不承担任何责任，因此此类包含和/或使用由客户自行承担风险。

NVIDIA 不作任何基于本文档的产品适用于任何特定用途的陈述或保证。NVIDIA 不一定对每种产品的所有参数进行测试。评估和确定本文档所含任何信息的适用性、确保产品适合并适用于客户计划的应用以及为应用执行必要的测试以避免应用或产品发生缺陷，完全由客户自行负责。客户产品设计中的缺陷可能影响 NVIDIA 产品的质量和可靠性，并可能导致本文档所含条件之外的其他或不同条件和/或要求。NVIDIA 对任何可能基于或归因于以下情况的缺陷、损害、成本或问题不承担任何责任：(i) 使用 NVIDIA 产品的方式违反本文档，或 (ii) 客户产品设计。

本文档未授予任何 NVIDIA 专利权、版权或其他 NVIDIA 知识产权的许可，无论是明示还是暗示。NVIDIA 发布的关于第三方产品或服务的信息不构成 NVIDIA 使用此类产品或服务的许可，也不构成对其的保证或认可。使用此类信息可能需要根据第三方专利或其他第三方知识产权获得第三方许可，或根据 NVIDIA 专利或其他 NVIDIA 知识产权获得 NVIDIA 许可。

复制本文档中的信息仅在经 NVIDIA 事先书面批准后方可进行，复制时不得更改，并完全遵守所有适用的出口法律和法规，并附带所有相关条件、限制和声明。

本文档和所有 NVIDIA 设计规范、参考板、文件、图纸、诊断、清单和其他文件（合称及单独称为"材料"）均按"原样"提供。NVIDIA 不对这些材料作任何明示、暗示、法定或其他方式的保证，并明确否认所有关于不侵权、适销性和特定用途适用性的暗示保证。在法律不禁止的范围内，在任何情况下，NVIDIA 对因使用本文档而产生的任何损害（包括但不限于任何直接、间接、特殊、附带、惩罚性或后果性损害）概不负责，无论其原因如何，也不论责任理论如何，即使 NVIDIA 已被告知此类损害的可能性。尽管客户可能因任何原因遭受任何损害，NVIDIA 对客户的有关本文所述产品的累计和责任总额应根据该产品的销售条款进行限制。

21.2 OpenCL

OpenCL 是 Apple Inc. 的商标，经许可由 Khronos Group Inc. 使用。

21.3 商标

NVIDIA 和 NVIDIA 徽标是 NVIDIA Corporation 在美国和其他国家的商标或注册商标。其他公司和产品名称可能是其各自关联公司的商标。

隐私政策 | 管理我的隐私 | 不出售或分享我的数据 | 服务条款 | 无障碍 | 公司政策 | 产品安全 | 联系方式

版权所有 © 2007-2026, NVIDIA Corporation 及其关联公司。保留所有权利。

最后更新于 2026 年 4 月 2 日。

> 原文出处: https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/
> 版权声明: 原文版权归 NVIDIA Corporation 所有，本翻译仅供学习参考。

[1]在具有集成 GPU 的片上系统（如 NVIDIA® Tegra®）上，主机内存和设备内存在物理上是相同的，但主机内存和设备内存之间仍然存在逻辑上的区分。详情请参见 CUDA for Tegra 的应用说明。